PROJECT REPORT

Explainable AI for Daily Short – Term Risk Prediction in Coinage Metal Markets

University of West London– School of Computing and Engineering

-

Final Year Project

Level 6

Due Date: 07/05/2026 23:59

-

Name: Mohammed Adnan Osman

Student ID: 33114153

-

Supervisor: Dr Nasim Dadashi

Module Code: CP60046E

Academic Year: 2025- 2026

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr Nasim Dadashi, for her continued guidance, support, and constructive feedback throughout this project. Her expertise and encouragement have been invaluable in shaping both the direction and quality of this research.

I would also like to thank the University of West London School of Computing and Engineering for providing the academic framework and resources that made this project possible.

Contents

Abstract

Machine learning has improved financial risk analysis, but many high-performing models remain opaque, which limits their use in high-stakes domains where interpretability is essential. This project addresses this challenge in gold, silver, and copper markets by developing an explainable machine learning system to predict downside risk events, defined as daily price declines exceeding two percent.

A comparative framework was implemented using a Random Forest classifier. A baseline model used technical and macroeconomic features, while an enhanced model also incorporated FinBERT sentiment scores from financial news. Data from 2020 to 2025 were stored in a PostgreSQL database, producing 4,525 trading records. Class imbalance was addressed using balanced class weighting, and a chronological split was used to prevent look-ahead bias.

The gold enhanced model achieved a ROC-AUC of 0.752, improving over the baseline score of 0.733. Silver also showed a ROC-AUC improvement, while copper demonstrated a substantial Brier Score improvement from 0.132 to 0.087. Backtesting on 2025 data confirmed the gold model generalizes well, achieving a ROC-AUC of 0.847. SHAP analysis confirmed that the models learned economically interpretable relationships, such as Treasury yields dominating gold predictions and S&P 500 returns dominating copper. Sentiment features did not appear in the top SHAP rankings due to sparse data coverage, which remains the primary limitation.

The findings show that interpretable machine learning can produce both practically useful risk scores and meaningful feature insights for commodity risk prediction. Future work should prioritize expanded sentiment coverage, threshold calibration for specific risk management needs, and the use of recurrent neural networks to better capture temporal dependencies.

1 Introduction

The integration of artificial intelligence into financial analytics has changed how market participants assess and manage risk. Machine learning can process large financial datasets, identify non-linear patterns, and generate predictive insights that often outperform traditional statistical approaches (Fischer and Krauss, 2018). However, many high-performing models remain opaque, which creates a problem in financial risk management because analysts must not only detect risk but also justify their assessments to stakeholders (Molnar, 2022).

Gold, silver, and copper occupy an important place in global financial markets. Gold and silver are often treated as safe-haven assets, while copper is more closely linked to industrial activity, infrastructure investment, and clean energy demand. Despite their significance, short-term downside risk prediction in these markets remains underexplored in the explainable AI literature, especially compared with research on equities, credit risk, and cryptocurrencies (Giudici et al., 2024).

This dissertation addresses three gaps in the literature. First, explainable AI has been applied only sparsely to commodity markets. Second, few studies have combined sentiment features with interpretable models in metal-market prediction. Third, many machine learning studies prioritise predictive accuracy over interpretability, which limits validation against financial fundamentals.

To address these gaps, this project develops an explainable AI system for predicting daily downside risk events in gold, silver, and copper. A downside risk event is defined as a next-day price decline exceeding two percent, selected as a meaningful threshold for risk management.

The research question is: Can a Random Forest classifier integrating FinBERT sentiment analysis accurately predict daily downside risk events in gold, silver, and copper markets while providing interpretable probability-based risk scores and SHAP explanations that align with established financial risk drivers? This reflects the need for both predictive performance and interpretability in responsible financial AI (Doshi-Velez and Kim, 2018). The report is structured as follows: Chapter 2 reviews the literature; Chapter 3 explains the methodology; Chapter 4 presents the system design and implementation; Chapter 5 reports the results and analysis; and Chapter 6 discusses the findings, limitations, and future work.

1.1 Aim and Objectives

1.1.1 Aim

The aim of this project is to develop an explainable machine learning system for predicting short-term downside risk events in coinage metal markets by integrating technical price indicators and macroeconomic features with NLP-derived market sentiment. The system generates probability-based daily risk scores supported by SHAP explanations, to enhance transparency and support informed financial risk analysis.

1.1.2 Objectives

1. Write a literature review examining the application of machine learning, explainable AI, and natural language processing in commodity and financial market risk prediction, identifying key research gaps.

2. Design and implement a structured data infrastructure, including a PostgreSQL database and automated collection pipelines for historical OHLCV price data, macroeconomic indicators, and FinBERT-processed financial news sentiment, covering the period 2020 to 2025.

3. Develop and validate a baseline Random Forest classifier trained exclusively on technical and macroeconomic features to predict daily downside risk events.

4. Develop an enhanced model incorporating FinBERT-derived sentiment features alongside technical and macroeconomic indicators and evaluate whether sentiment integration improves predictive performance relative to the baseline.

5. Apply SHAP-based explainability to both models to quantify the contribution of individual features to risk predictions and assess the interpretability of model outputs in relation to established financial risk drivers.

6. Evaluate model calibration quality using probabilistic metrics including Brier Score and Log Loss to ensure that predicted risk probabilities are reliable and trustworthy for practical risk management applications.

7. Conduct walk-forward back testing on unseen 2025 market data and generate SHAP waterfall plots for notable historical risk events to validate model explanations against known market conditions and economic fundamentals.

8. Document and critically evaluate the findings, limitations, and implications of the research, providing recommendations for the future application of transparent and explainable AI in financial risk analysis.

2 Literature Review

2.1 Artificial intelligence and Machine Learning in Financial Risk Prediction

Machine learning has become increasingly important in financial risk prediction over the past decade, largely because traditional statistical models struggle with large, complex financial datasets. Classical approaches such as ARIMA and GARCH rely on assumptions of linearity and stationarity that are often violated in real markets, where returns are non-linear, heavy-tailed, and subject to structural breaks (Fischer and Krauss, 2018). Fischer and Krauss (2018) showed that LSTM networks can outperform traditional econometric benchmarks in daily stock return prediction, but they also highlighted a major limitation: deep learning models are often difficult to interpret.

Random Forest classifiers offer a strong alternative because they combine good predictive performance with greater interpretability. By building multiple decorrelated trees and aggregating their outputs, Random Forests reduce the overfitting risk associated with single decision trees and work well with noisy, high-dimensional financial data (Peng and Yan, 2021). A review by Peng and Yan (2021) found that ensemble methods performed competitively across tasks such as credit risk, volatility forecasting, and fraud detection, while also offering a better balance between accuracy and transparency than deep neural networks.

Class imbalance is another major challenge in financial risk prediction. Adverse events such as large daily price declines occur much less often than normal events, so a model can appear accurate while still missing the cases that matter most. To address this, class-weighted training can increase the penalty for misclassifying minority risk events and improve recall without a major loss in overall performance (Peng and Yan, 2021). This approach directly informs the methodology used in this study, where balanced class weighting is applied during Random Forest training to reduce the chance of overlooking downside risk events.

2.2 Explainable AI and Model Interpretability in Finance

Explainable AI has become important because predictive accuracy alone is not enough in high-stakes areas such as finance, healthcare, and law. Doshi-Velez and Kim (2018) argued that explanation is needed whenever the problem is not fully specified and the cost of a wrong decision is high. In financial risk management, this is especially relevant because analysts must justify risk assessments, meet regulatory expectations, and align model

outputs with economic fundamentals. Black-box models therefore remain difficult to adopt in practice (Monica Hovsepian, 2023).

SHAP (SHapley Additive Explanations), introduced by Lundberg and Lee (2017), is now one of the most widely used methods for post-hoc explanation in finance. It is based on cooperative game theory and calculates the average contribution of each feature to a prediction. SHAP is popular because it provides local explanations, global feature importance, and works with many model types. A review by Danie et al. (2025) of 150 studies found SHAP was the most commonly used explainability method in financial XAI research.

Jabeur, Mefteh-Wali and Viviani (2024) applied SHAP with XGBoost to gold price forecasting and found that macroeconomic variables such as the US Dollar Index, crude oil prices, and Treasury yields were the most influential features. Their results supported established economic theory about gold as a safe-haven asset. Their study is relevant here because it also combines machine learning with SHAP, although it focuses on price forecasting rather than downside risk classification and does not include NLP sentiment features.
Giudici et al. (2024) found that most XAI research in finance focuses on equity markets, with much less attention given to commodities and metals. They also showed that commodity markets have different drivers, including supply-side and industrial demand factors, so models should be tested specifically in this context. Zhang, Liang and Ou (2024) reinforced this view by showing that silver and copper predictions were influenced more by cross-metal correlations and industrial demand than gold predictions.

SHAP is powerful, but it is not perfect. To reduce computational cost while keeping theoretical consistency, this study uses TreeExplainer, which is designed for tree-based models such as Random Forest.

2.3 Natural Language Processing and Sentiment Analysis in Market Prediction

The use of textual sentiment in financial forecasting has a long research history. Early work by Bollen, Mao and Zeng (2011) showed that Twitter mood indicators could predict movements in the Dow Jones Industrial Average up to three days ahead, suggesting that public sentiment can act as a useful market signal. Although Twitter-based methods have since been replaced by more advanced approaches, the core idea remains: unstructured text can contain predictive information that price data alone may miss.

Transformer-based language models, especially BERT, improved sentiment analysis by capturing context rather than relying on simple word lists. Lexicon-based methods often miss meaning because the same word can imply different sentiment depending on context. FinBERT, which is pre-trained on financial text such as news, earnings reports, and analyst commentary, addresses this limitation by learning finance-specific language patterns (Araci, 2019; Liu et al., 2020). Huang, Wang and Yang (2022) found that FinBERT outperformed general-purpose BERT on financial text classification, particularly when handling financial jargon and context-dependent language.

Recent studies have also shown that FinBERT features can improve financial prediction models. Gu et al. (2024) found that adding FinBERT sentiment to an LSTM model improved stock prediction accuracy compared with using price data alone. Similarly, a FinBERT-XGBoost model with SHAP explanations outperformed both a technical-only baseline and a lexicon-based sentiment model, achieving an AUC of 0.748 versus 0.593 for the technical-only model (Mathematics, 2025). In that study, SHAP showed that FinBERT-derived features were especially important during volatile periods such as the COVID-19 market shock, where sentiment captured fear and news-driven risk more effectively than price features. This supports the present study's hypothesis that sentiment may improve downside risk prediction during stressed market conditions.

However, the effect of sentiment is not consistent across all studies. Li et al. (2020) found that sentiment can improve financial prediction, but the size of the benefit depends on the asset class, sentiment source, and forecast horizon. Liapis and Karanikola (2023) also reported that FinBERT was not consistently better than simpler lexicon-based methods across all markets and time periods. This suggests that FinBERT's value is context-dependent rather than universal. For that reason, the present study compares a baseline model without sentiment against an enhanced model using FinBERT, allowing the contribution of sentiment to be tested directly in coinage metal downside risk prediction. The three metals in this study also present different prediction challenges. Gold behaves mainly as a macroeconomic hedge influenced by interest rate expectations and geopolitical risk. Silver has both monetary and industrial characteristics, making its behaviour more complex. Copper is closely tied to global growth expectations and equity market conditions. As a result, the model must generalise across three different risk mechanisms, which makes this a more demanding test than studies focused on a single asset class.

2.4 Downside Risk Prediction and Threshold-Based Classification

Financial risk prediction is often framed as a binary classification problem, where the model predicts whether an adverse event will occur on the next trading day. This approach

is useful because it produces clear outputs, is less affected by short-term noise than regression-based forecasting and can be evaluated with metrics that are directly relevant to risk management (Peng and Yan, 2021). The threshold used to define a risk event is also important because it affects both how common the target class is and how economically meaningful the event is. In this study, a two percent daily price decline is used because it is a practical threshold for identifying significant downside movements in commodity markets.

Evaluating binary risk models requires more than accuracy, especially when risk events are rare. Recall is particularly important because missing a real risk event is usually worse than raising a false alarm. ROC-AUC measures how well the model separates risk from non-risk cases, while Brier Score and Log Loss assess whether the predicted probabilities are well calibrated. Using these metrics together provides a more complete comparison between the baseline and enhanced models than accuracy alone (Doshi-Velez and Kim, 2018).

2.5 Summary and Research Gaps

The literature shows that machine learning is now an established approach to financial risk prediction, with Random Forests offering a useful balance between predictive performance and interpretability. SHAP has become a leading method for explaining model outputs in finance and has already been applied to gold price forecasting and broader financial risk analysis. FinBERT-based sentiment analysis has also been shown to improve financial prediction by capturing information not present in numerical market data, although results vary across asset classes and market conditions.

Despite this progress, three research gaps remain. First, explainable AI has been only lightly applied to downside risk prediction in coinage metal markets such as gold, silver, and copper, even though these markets are influenced by supply-side and geopolitical factors that differ from equity or credit markets. Second, few studies have combined FinBERT sentiment features and explainable machine learning in a single framework for metal market risk prediction. Third, many studies treat accuracy and interpretability as competing goals, whereas this project treats them as complementary by using SHAP to explain predictions and check whether they align with financial theory.

These gaps justify the present study. By combining an interpretable Random Forest model with FinBERT sentiment features and SHAP explanations, this dissertation addresses an underexplored area in explainable AI and commodity risk prediction.

3 Research Methodology

This chapter details the research approach adopted in this project, explaining every methodological decision in sufficient depth for the study to be fully replicated. The methodology is structured around five interconnected stages: research design, data collection, feature engineering, model development, and evaluation. Each stage is described in terms of what was done, why it was done, and how it relates to the overall research question and objectives.

3.1 Research Design

This project uses a quantitative, experimental design. It tests hypotheses about the value of sentiment analysis and the interpretability of machine learning outputs through experiments on real financial data, with results measured against performance criteria. The study follows a positivist approach because it relies on observable, measurable phenomena and aims for findings that can be replicated and generalised (Doshi-Velez and Kim, 2018).

The methodology uses a comparative design with two models trained under the same conditions. The baseline model is a Random Forest classifier using technical price indicators and macroeconomic features only. The enhanced model uses the same features plus FinBERT sentiment scores extracted from financial news headlines. By keeping all other variables constant and changing only the sentiment input, the design isolates the effect of NLP-based sentiment analysis on predictive performance. This directly supports Objective 2 and Objective 4 and provides a clear basis for answering the research question.

3.2 Data Collection

3.2.1 Price and Macroeconomic Data

Historical OHLCV (Open, High, Low, Close, Volume) price data for gold (ticker: GC=F), silver (SI=F), and copper (HG=F) were collected covering the period from January 2020 to December 2025, providing approximately 1,500 trading days per metal. Data were retrieved using the yFinance Python library, which provides programmatic access to Yahoo Finance market data.

Four macroeconomic indicators were collected via yFinance: the US Dollar Index, VIX, 10-year Treasury yield, and S&P 500 daily return

3.2.2 Financial News Sentiment Data

Financial news headlines related to gold, silver, and copper markets were collected via the NewsAPI service, which aggregates headlines from major financial news sources including Reuters, Bloomberg, Financial Times, and MarketWatch. Queries were constructed using metal-specific search terms — for example, "gold price", "silver market", and "copper commodity" — to ensure relevance to the target asset classes. GDELT was initially evaluated as an alternative news source but was found to be unreliable for production pipelines due to inconsistent data availability and quality issues, and was therefore replaced by NewsAPI.

Raw headline text was processed through the FinBERT model — a BERT-based language model pre-trained on financial text corpora (Araci, 2019; Liu et al., 2020) — to generate three probability scores for each headline: positive, negative, and neutral sentiment.

3.3 Data Preprocessing and Feature Engineering

3.3.1 Target Variable Construction

The binary target variable represents whether a downside risk event occurs on the next trading day. For each day t, the label is set to 1 if the return on day t is below negative two percent, and 0 otherwise. This forward-looking setup prevents look-ahead bias because the model uses only information available at day t to predict day t . The two percent threshold was chosen because it captures economically meaningful declines in commodity markets while still producing enough positive cases for training.

3.3.2 Technical Indicators

A comprehensive set of technical indicators was derived from OHLCV price data to capture momentum, trend, volatility, and volume. These were selected for their prominence in financial literature and their relevance to short-term price risk. The features engineered for each metal include:

- Simple Moving Averages (SMA): 5, 10, 20, and 50-day windows for multi-horizon trend analysis.

- Simple Moving Averages (SMA): 5, 10, 20, and 50-day windows for multi-horizon trend analysis.

- Relative Strength Index (RSI): 14-day window to measure the speed and magnitude of price changes.

- Moving Average Convergence Divergence MACD: The difference between 12 and 26-day EMAs, including the signal line and histogram.

- Bollinger Bands: 20-day window including bandwidth and percentage B to measure volatility and relative price position.

- Average True Range (ATR) at a 14-day window, measuring market volatility by decomposing the entire range of an asset price for a given period.

- 20-day historical volatility, computed as the rolling standard deviation of log returns.

- Daily return, log return, high-low ratio, volume change, and 20-day volume moving average as additional price and volume features.

Finally, lag features at t = 1, t = 2, and t = 3 were computed for key variables to provide a short-term historical window and prevent look-ahead bias.

3.4 Model Development

3.4.1 Algorithm Selection

Random Forest was selected as the classification algorithm for both models due to four key factors. First, it effectively models the non-linear relationships common in financial data. Second, its ensemble structure makes it robust to the outliers and noise typical of financial time series. Third, it is directly compatible with SHAP TreeExplainer, which provides computationally efficient and theoretically exact feature explanations (Lundberg and Lee, 2017). Finally, Random Forests are well-established in financial risk research, providing a reliable benchmark for evaluating sentiment feature contributions (Peng and Yan, 2021).

3.4.2 Handling Class Imbalance

Since downside risk events (daily declines exceeding two percent) represent a minority class (8–12% of the data), class imbalance poses a significant modelling challenge. A naive model predicting "no risk" would achieve high overall accuracy while failing to detect actual risk events. To address this, the study uses scikit-learn's balanced class weight formula, which sets weights inversely proportional to class frequency. This approach penalises misclassifications of risk events more heavily than non-events, improving recall for the minority class without the need to discard training data.

3.4.3 Train, Validation and Test Split

The dataset for each metal was divided into training, validation, and test sets using a chronological 60/20/20 split. This means the earliest sixty percent of trading days form the training set, the next twenty percent form the validation set used for hyperparameter tuning, and the final twenty percent form the held-out test set used for final evaluation. Crucially, this split is strictly chronological — no random shuffling is applied — preserving the temporal order of the data and preventing look-ahead bias.

3.4.4 Hyperparameter Tuning

Hyperparameter optimization was performed using GridSearchCV with TimeSeriesSplit cross-validation, configured with five folds. The best hyperparameter combination was selected based on ROC-AUC score on the validation folds, as ROC-AUC is robust to class imbalance and directly measures the model's ability to discriminate between risk and non-risk days across all possible classification thresholds.

3.5 Explainability Framework

SHAP (SHapley Additive ExPlanations) was applied to both the baseline and enhanced models to generate interpretable explanations of their predictions. The TreeExplainer variant was used, which is specifically optimised for tree-based models and computes exact Shapley values rather than approximations, ensuring that explanations are theoretically consistent and faithful to the model's actual decision-making process (Lundberg and Lee, 2017).

3.6 Evaluation Framework

Model performance was assessed using classification and probabilistic metrics to balance accurate event detection with reliable probability estimation. Probabilistic calibration was evaluated using the Brier Score, which measures the mean squared difference between predictions and outcomes, and Log Loss, which penalises overconfident errors. Well-calibrated probabilities are essential in risk management, where analysts must rank days by risk level and base decisions on the magnitude of the score rather than binary labels. Calibration curves were also generated to visually assess the alignment between predicted probabilities and observed event frequencies.

3.8 Limitations of Methodology

Several methodological limitations should be acknowledged. First, the NewsAPI free tier limits historical data access, resulting in sparse sentiment coverage for earlier periods (2020–2025). Days with missing headlines are assigned a neutral sentiment value (zero), which may underestimate the predictive contribution of sentiment if data availability is non-random (e.g., lower headline volume during low-volatility periods).

Second, financial markets are non-stationary, meaning feature-target relationships may shift over time. While walk-forward backtesting on 2025 data evaluates the model on recent information, it cannot guarantee future performance. Model outputs should therefore be treated as analytical indicators rather than definitive forecasts, and any production implementation would require periodic retraining to remain accurate.

4 Design and Implementation

This chapter documents the design and implementation of the six-stage pipeline, covering the database schema, data collection, feature engineering, model training, evaluation, and SHAP explainability integration.

4.1 System Architecture Overview

The system is structured as a sequential six-stage data pipeline, with each stage implemented as a standalone numbered Python script. This design was deliberately chosen to separate concerns clearly, making each component independently testable and maintainable. The pipeline progresses through six stages: data collection, feature engineering, sentiment scoring, model training, evaluation, and backtesting.

4.2 Database Design

The PostgreSQL database metal_risk_prediction was designed with a normalized relational schema to avoid data duplication and ensure referential integrity across all tables. The scheme consists of seven tables, described below.

Table 1: PostgreSQL database schema summary

Table	Primary Key	Description
metals	metal_id (INT)	Reference table storing the three metals: gold (1), silver (2), copper (3).
Price_data	price_id (SERIAL)	Daily OHLCV price data for each metal. UNIQUE constraint on (metal_id, date).
Technical_features	feature_id (SERIAL)	All engineered technical indicators per metal per day. NUMERIC(20,6)

		precision throughout to avoid overflow.
Macroeconomic_data	macro_id (SERIAL)	Daily values for DXY, VIX, 10-year Treasury yield, and S&P 500. Not metal-specific — shared across all three metals.
Sentiment_data	sentiment_id (SERIAL)	Individual headline sentiment scores from FinBERT, including positive, negative, and neutral probability scores per headline.
Daily_sentiment	daily_sent_id (SERIAL)	Daily aggregated sentiment features per metal: mean scores, headline count, positive/negative ratios, and sentiment standard deviation.

All NUMERIC columns in the technical_features table were defined with precision NUMERIC(20,6) to prevent overflow errors that were encountered during development when storing values such as volume-based indicators that can span multiple orders of magnitude.

4.2.1 Schema Structure and Table Design

The database comprises six tables. The price_data table stores OHLCV data, with CHECK constraints ensuring all price and volume values are non-negative, rejecting invalid data before the feature engineering pipeline. The macroeconomic_data table is market-wide rather than metal-specific, avoiding redundant storage by joining at query time. The daily_sentiment and sentiment_data tables store FinBERT-scored headlines and their daily aggregations, as detailed in Section 4.2.4. The two primary tables, metals and technical_features, are described below.

Metals

```
-- 1) METALS
CREATE TABLE IF NOT EXISTS metals (
  metal_id SERIAL PRIMARY KEY,
```

```
  symbol VARCHAR(10) UNIQUE NOT NULL,
  name VARCHAR(50) NOT NULL,
  -- ... (full schema: database/schema.sql)
);
```

The metals table stores the three metal identifiers once, referenced by all time-series tables via metal_id rather than repeating metal names across thousands of rows. The market_type column distinguishes precious metals from industrial metals, which is analytically relevant since gold and silver behave differently to copper as risk assets. The INSERT statement uses ON CONFLICT DO NOTHING so seed data can be re-inserted safely without causing duplicate key errors during development.

Technical features

```
-- 4) TECHNICAL FEATURES
CREATE TABLE IF NOT EXISTS technical_features (
  feature_id SERIAL PRIMARY KEY,
  metal_id   INTEGER NOT NULL REFERENCES metals(metal_id) ON DELETE CASCADE,
  date      DATE NOT NULL,
  daily_return NUMERIC(18, 10),
  log_return   NUMERIC(18, 10),
  rsi_14 NUMERIC(6, 2),
  -- ... MACD, Bollinger, SMA, EMA, volume columns
  -- (full schema: database/schema.sql)
  UNIQUE (metal_id, date)
);
```

Three precision levels were used depending on column type. RSI uses NUMERIC(6,2) because its output is always bounded between 0 and 100. Return and ratio columns use NUMERIC(18,10) because they produce small decimal values requiring high precision. Moving average and price-level columns use NUMERIC(20,6) because they store actual price values that can span a wide range — gold futures trade above 2,000 USD — requiring both a large integer part and sufficient decimal precision. The UNIQUE constraint on (metal_id, date) prevents duplicate daily records, and ON DELETE CASCADE automatically removes feature records if a metal is deleted from the reference table.

4.2.2 Precision and Data Integrity Decisions

During development a numeric overflow error was encountered when inserting volume-based indicators into the technical_features table. The error was reproduced and diagnosed using pgAdmin as shown in Figure below.

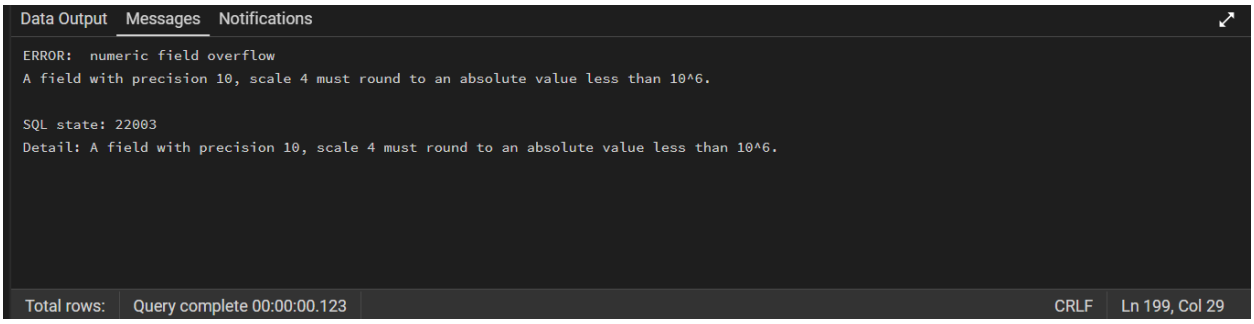


Figure 1: PostgreSQL numeric field overflow error encountered when inserting volume-based indicator values using insufficient column precision NUMERIC(10,4).

The error confirms that NUMERIC(10,4) supports a maximum absolute value less than 10^6, meaning any volume-based indicator exceeding 999,999 would cause the entire batch insertion to fail. Gold futures volume values regularly exceed this range during high-activity trading sessions, causing the pipeline to crash silently on those dates.

4.2.3 Idempotent Insertion Strategy

All six data insertion scripts were designed to be idempotent — they can be re-run multiple times without creating duplicate records. The insertion function for price data is shown below in 01_data_collection.

```
def insert_price_data(conn, metal_id, df):
  records = [(metal_id, r["date"], float(r["open"]), ...)
        for _, r in df.iterrows()]
  sql = """INSERT INTO price_data (...) VALUES %s
       ON CONFLICT (metal_id, date) DO NOTHING;"""
  # ... (full implementation: scripts/01_data_collection.py)
  execute_values(cur, sql, records, page_size=2000)
```

The ON CONFLICT DO NOTHING clause ensures the script can be re-run safely at any point without requiring the database to be manually cleared and execute_values sends the entire batch in a single database call for efficiency.

4.2.4 Sentiment Feature Aggregation

The sentiment pipeline stores individual FinBERT-scored headlines in the sentiment_data table, but the Random Forest model requires a single set of daily features per metal rather

than a variable number of individual headline scores. The aggregation from raw headlines to daily model-ready features is performed entirely in SQL, as shown below in 03_sentiment_pipeline.

```
INSERT INTO daily_sentiment (metal_id, date, avg_sentiment,
  avg_positive, avg_negative, avg_neutral, headline_count,
  positive_ratio, negative_ratio, sentiment_std)
SELECT metal_id, date,
  AVG(sentiment_score), AVG(positive_score),
  -- ... (full query: scripts/03_sentiment_pipeline.py)
FROM sentiment_data WHERE metal_id = :metal_id
GROUP BY metal_id, date
ON CONFLICT (metal_id, date) DO UPDATE SET ...
```

The aggregation produces eight daily features from the raw headline scores. AVG(sentiment_score) computes the mean sentiment across all headlines published on a given day, producing a value between -1 and +1. The positive_ratio and negative_ratio columns use a CASE statement to convert categorical sentiment labels into proportions — for example a positive_ratio of 0.7 means 70% of that day's headlines were classified as positive by FinBERT.

4.3 Data Collection Pipeline

4.3.1 Script 01: Price and Macroeconomic Data Collection

Script 01 collects historical OHLCV price data and macroeconomic indicators via the yFinance API and stores them in the PostgreSQL database using idempotent batch insertions.

The script consists of seven functions:

- get_metal_map — retrieves metal identifiers from the database
- _flatten_yfinance_columns — handles yFinance MultiIndex column inconsistency
- download_prices — downloads and cleans OHLCV price data
- download_macro — downloads four macroeconomic indicators
- insert_price_data — inserts price data using batch insertion
- verify_counts — post-insertion verification query
- main — orchestrates the full pipeline

get_metal_map retrieves metal identifiers and ticker symbols dynamically from the metals table rather than hardcoding them, ensuring the script adapts automatically to any changes in the database configuration. verify_counts runs a post-insertion query immediately after all data has been committed, printing the total row count and date range for each metal to confirm the pipeline completed successfully without requiring manual database inspection.

The two most technically significant functions, download_prices and download_macro, are documented in full below using the software development lifecycle framework covering requirements, design, implementation, and testing.

download_prices

Requirements

The download_prices function must retrieve historical daily OHLCV price data for a given metal ticker from Yahoo Finance covering January 2020 to December 2025. It must handle inconsistencies in yFinance column output across library versions and return a clean DataFrame ready for database insertion.

Design

```
FUNCTION download_prices(ticker, start, end):
  Download OHLCV data from yFinance
  IF empty: RETURN None

  Flatten MultiIndex columns
  Standardise date and column names to lowercase
  Validate all required columns exist
  Remove rows with missing price values
  RETURN cleaned DataFrame
END FUNCTION
```

Implementation

```
def download_prices(ticker, start="2020-01-01", end="2025-12-31"):
  df = yf.download(ticker, start=start, end=end,
            progress=False, auto_adjust=False)
  if df is None or df.empty:
    return None
```

```
df = _flatten_yfinance_columns(df)
df = df.reset_index()
# ... column standardisation and validation (see scripts/01_data_collection.py)
df["date"] = pd.to_datetime(df["date"]).dt.date
return df[["date","open","high","low","close","volume","adjusted_close"]]
```

Testing

he function was validated through the pipeline run shown in Figures 1 and 2. Successful insertion of 1,509 rows for gold, 1,509 for silver, and 1,510 for copper confirms clean DataFrames were returned for all three tickers across the full 2020-2025 window.

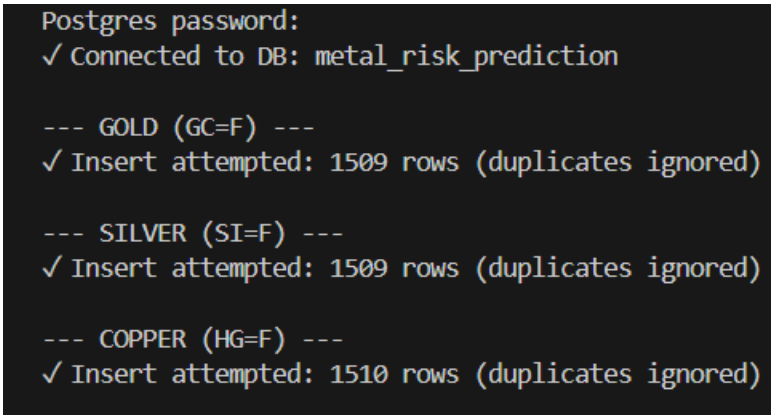


Figure 2: Terminal output confirming successful OHLCV price data insertion for gold (1,509 rows), silver (1,509 rows), and copper (1,510 rows) via the download_prices function in Script 01.

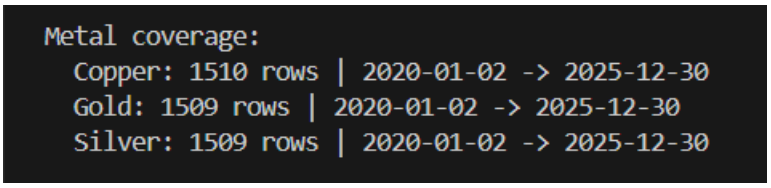


Figure 3: Terminal output from verify_counts confirming full date range coverage for all three metals from 2020-01-02 to 2025-12-30.

download_macro

Requirements

The download_macro function must download daily values for four macroeconomic indicators and merge them into a single DataFrame aligned by date. Missing values must be handled using forward-fill imputation and S&P 500 daily return computed as a derived feature.

Design

FUNCTION download_macro(start, end):

```
  Define four tickers: DXY, VIX, TNX, S&P500

  FOR each ticker:
    Download closing price from yFinance
    Store as named column (usd_index, vix, etc.)

  Merge all four DataFrames on date using outer join
  Sort by date
  Forward-fill any missing values
  Compute sp500_return as daily percentage change
  Remove rows where any core indicator is missing
  RETURN cleaned DataFrame
END FUNCTION
```

Implementation

```
def download_macro(start="2020-01-01", end="2025-12-31"):
  tickers = {"DX-Y.NYB": "usd_index", "^VIX": "vix",
        "^TNX": "treasury_yield_10y", "^GSPC": "sp500_close"}
  frames = []
  for tkr, col in tickers.items():
    df = yf.download(tkr, start=start, end=end, progress=False)
    # ... column flattening and renaming
    frames.append(df[["date", col]])
  macro = frames[0]
  for f in frames[1:]:
    macro = macro.merge(f, on="date", how="outer")
  macro = macro.sort_values("date").ffill()
  macro["sp500_return"] = macro["sp500_close"].pct_change()
  return macro.dropna(subset=["usd_index","vix","treasury_yield_10y",
                "sp500_close","sp500_return"])
```

Testing

The function was validated through the pipeline run shown in Figure 3. Insertion of 1,508 macroeconomic rows confirms all four indicators were downloaded, merged, and cleaned correctly. The slightly lower row count compared to price data reflects the dropna removing days where any macro indicator was unavailable.

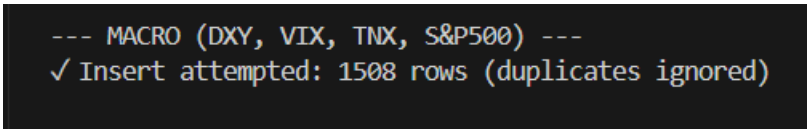


Figure 4: Terminal output confirming successful macroeconomic data insertion of 1,508 rows covering the US Dollar Index, VIX, 10-year Treasury yield, and S&P 500 via the download_macro function in Script 01.

4.3.2 Script 02: Feature Engineering

Script 02 reads raw OHLCV data from the database, engineers all technical indicators required as model input features, and stores the computed features back into the technical_features table.

- create_db_connection — establishes the SQLAlchemy database connection
- calculate_rsi — computes the Relative Strength Index
- calculate_macd — computes the Moving Average Convergence Divergence
- calculate_bollinger — computes Bollinger Bands and bandwidth
- load_price_data — loads raw OHLCV data from the database for one metal
- build_features — engineers all technical indicators from price data
- build_risk_events — constructs the binary risk event labels
- upsert_technical_features — inserts engineered features into the database
- main — orchestrates the full feature engineering pipeline

calculate_macd computes the difference between the 12-day and 26-day exponential moving averages alongside the signal line and histogram. calculate_bollinger computes the upper, middle, and lower bands at two standard deviations alongside the bandwidth ratio.

Calculate_rsi

Requirements

The calculate_rsi function must compute the Relative Strength Index using Wilder's smoothing method over a 14-day period, returning values bounded between 0 and 100 that are consistent and comparable across all three metals.

Design

FUNCTION calculate_rsi(series, period):
  Compute daily price changes
  Separate changes into gains and losses
  Compute rolling mean of gains over period
  Compute rolling mean of losses over period
  Compute RS = average gain / average loss

```
  RETURN 100 - (100 / (1 + RS))
END FUNCTION
```

Implementation

```
def calculate_rsi(series: pd.Series, period=14):
  delta = series.diff()
  gain = delta.where(delta > 0, 0).rolling(period).mean()
  loss = (-delta.where(delta < 0, 0)).rolling(period).mean()
  rs = gain / loss
  return 100 - (100 / (1 + rs))
```

Testing

The function was validated through Script 02 shown in Figure 5. Insertion of valid feature rows for all three metals with no overflow errors confirms RSI values were correctly bounded between 0 and 100, consistent with the NUMERIC(6,2) constraint.

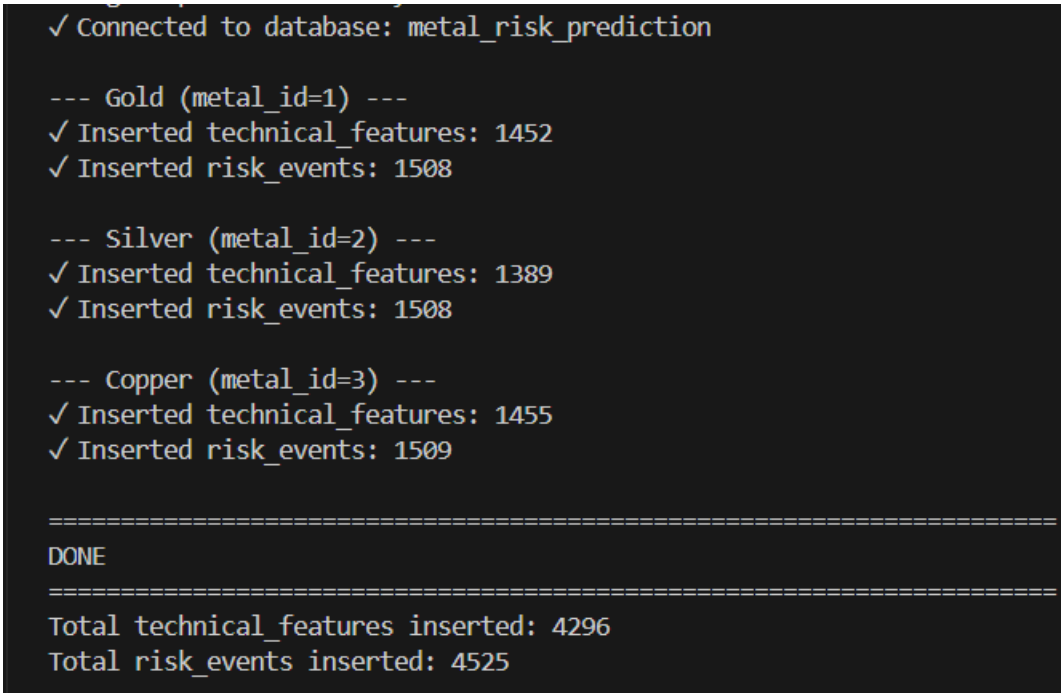


Figure 5: Terminal output confirming successful feature engineering and risk event labelling for all three metals via Script 02, with 4,296 technical feature rows and 4,525 risk event rows inserted into the PostgreSQL database.

Built_features

Requirements

The build_features function must compute all technical indicators required as model input features from the raw OHLCV data. It must handle infinite values produced by ratio calculations before returning the DataFrame to prevent database insertion failures.

Design

```
FUNCTION build_features(df):
  Compute daily return and log return from close price

  Compute SMAs at 5, 10, 20, 50 day windows
  Compute EMAs at 12 and 26 day windows

  Compute Bollinger Bands (upper, middle, lower, width)
  Compute RSI at 14 day window
  Compute MACD, signal line and histogram

  Compute high-low range and high-low ratio
  Compute volume change and 20 day volume moving average

  Replace all inf and -inf values with NaN
  RETURN DataFrame with all features
END FUNCTION
```

Implementation

```
def build_features(df: pd.DataFrame):
  df = df.copy()
  df["daily_return"] = df["close"].pct_change()
  df["log_return"]   = np.log(df["close"] / df["close"].shift(1))
  df["sma_5"]      = df["close"].rolling(5).mean()
  # ... sma_10, sma_20, sma_50, ema_12, ema_26, Bollinger,
  # ... RSI, MACD, high-low features, volume features
  # (full implementation: scripts/02_feature_engineering.py)
  df.replace([np.inf, -np.inf], np.nan, inplace=True)
  return df
```

Testing

The function was validated through Script 02 shown in Figure 5. Insertion of 4,296 technical feature rows across all three metals confirms all indicators were produced correctly with

no overflow errors. The inf/-inf replacement was specifically tested on the volume_change column, confirming the fix prevents PostgreSQL rejection of rows where previous-day volume was zero.

4.3.3 Script 03: FinBERT Sentiment Pipeline

Script 03 collects financial news headlines from NewsAPI, scores each headline using the FinBERT sentiment model, aggregates the scores to daily level, and stores the results in the PostgreSQL database.

- create_db_connection — establishes the SQLAlchemy database connection
- FinBERTAnalyzer — loads and runs the FinBERT model for sentiment scoring
- collect_newsapi_headlines — fetches headlines from NewsAPI for a given keyword
- collect_headlines_for_metal — collects and deduplicates headlines for all keywords of a metal
- insert_sentiment_data — inserts individual scored headlines into the database
- aggregate_daily_sentiment — aggregates headline scores to daily features
- main — orchestrates the full sentiment pipeline

collect_newsapi_headlines queries the NewsAPI endpoint using metal-specific search terms and returns a list of headline dictionaries with date, source, and text. collect_headlines_for_metal iterates over all keywords for a given metal, deduplicating headlines across keyword queries to avoid scoring the same headline multiple times.

FinBERTAnalyzer

Requirements

The FinBERTAnalyzer class must load the ProsusAI/finbert model once at startup and process headlines in batches of 32, producing positive, negative, and neutral probability scores and a composite sentiment score on the interval -1 to +1.

Design

```
CLASS FinBERTAnalyzer:
  INIT:
    Load FinBERT tokenizer and model from HuggingFace
    Set model to evaluation mode

  FUNCTION analyze(headlines):
    FOR each batch of 32 headlines:
```

```
        Tokenize batch with padding and truncation
        Run model forward pass with no gradient computation
        Apply softmax to get probabilities
        FOR each headline in batch:
            Extract positive, negative, neutral probabilities
            Compute sentiment score = positive - negative
            Store result
    RETURN all results
END CLASS
```

Implementation

```
class FinBERTAnalyzer:
  def __init__(self):
    self.tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert")
    self.model     = AutoModelForSequenceClassification.from_pretrained(
                  "ProsusAI/finbert")
    self.model.eval()

  def analyze(self, headlines: list) -> list:
    results = []
    for i in range(0, len(headlines), 32):
      batch  = headlines[i:i+32]
      inputs = self.tokenizer(batch, padding=True, truncation=True,
                    max_length=128, return_tensors="pt")
      with torch.no_grad():
        probs = torch.nn.functional.softmax(
          self.model(**inputs).logits, dim=-1)
      # ... extract pos/neg/neu scores and build results list
      # (full implementation: scripts/03_sentiment_pipeline.py)
    return results
```

Testing

The pipeline was validated through Script 03 shown in Figures 6 and 7. The model loaded successfully and scored all 1,321 headlines producing 346 negative, 692 neutral, and 283 positive classifications. The dominance of neutral classifications is consistent with expected financial news behaviour.

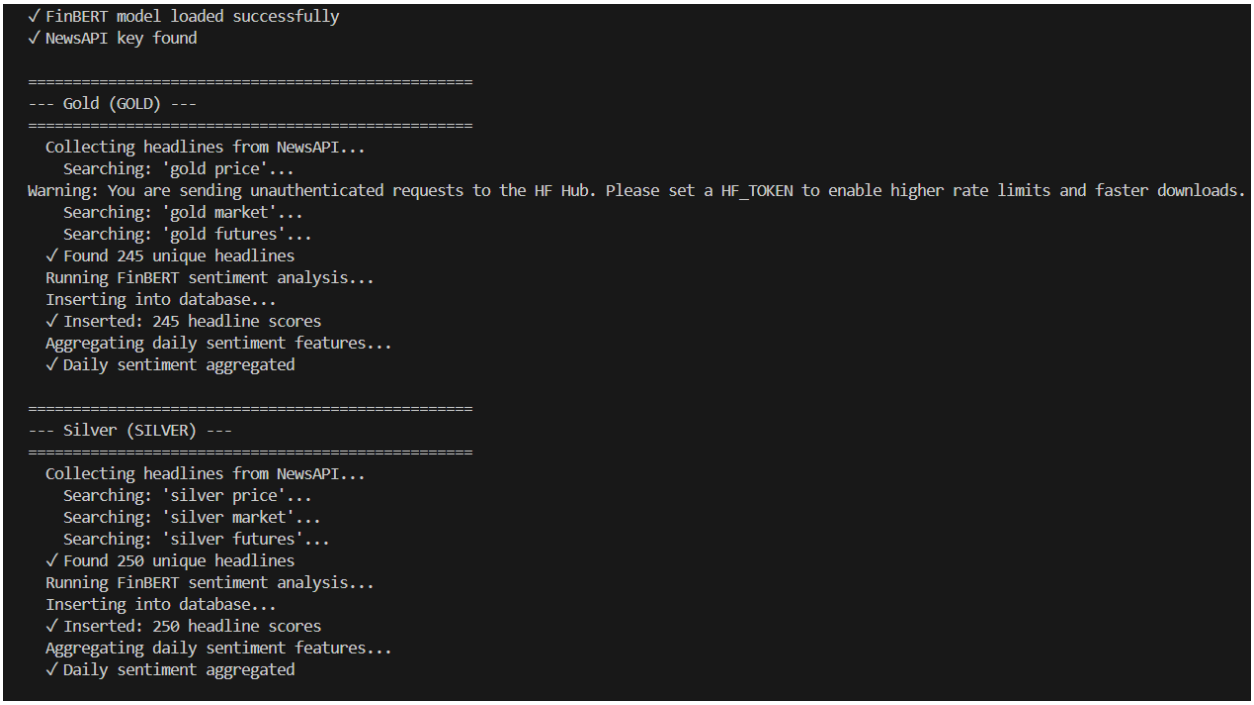


Figure 6: Terminal output confirming FinBERT model loading and successful headline collection and sentiment scoring for gold (245 headlines) and silver (250 headlines) via Script 03.

```
================================================
--- Copper (COPPER) ---
================================================
  Collecting headlines from NewsAPI...
    Searching: 'copper price'...
    Searching: 'copper market'...
    Searching: 'copper futures'...
  ✓ Found 202 unique headlines
  Running FinBERT sentiment analysis...
  Inserting into database...
  ✓ Inserted: 202 headline scores
  Aggregating daily sentiment features...
  ✓ Daily sentiment aggregated

================================================================
VERIFICATION
================================================================
  Total headlines scored:    1321
  Daily sentiment records:   168
  Copper: 410 headlines | 56 days | 2026-01-27 to 2026-03-24
  Gold: 446 headlines | 56 days | 2026-01-27 to 2026-03-24
  Silver: 465 headlines | 56 days | 2026-01-27 to 2026-03-24

  Sentiment distribution:
    negative: 346
    neutral: 692
    positive: 283

✓ SENTIMENT PIPELINE COMPLETE
```

Figure 7: Terminal output confirming successful sentiment scoring for copper (202 headlines) and verification counts showing 1,321 total headlines scored across 168 daily sentiment records, with sentiment distribution of 346 negative, 692 neutral, and 283 positi

4.4 Model Training Implementation

4.4.1 Script 04: Model Training

Script 04 loads all feature data from the database, constructs the binary target variable, splits the data chronologically, trains both baseline and enhanced Random Forest models, generates SHAP explanations, and saves all trained models and outputs to disk.

- get_engine — establishes the SQLAlchemy database connection
- load_feature_matrix — loads and merges technical, macroeconomic, and sentiment features from the database
- build_target_and_lag_features — constructs the binary target variable and lag features
- get_feature_sets — defines baseline and enhanced feature column lists

- temporal_split — divides the dataset chronologically into training, validation, and test sets
- train_random_forest — trains the Random Forest classifier with hyperparameter tuning
- evaluate_model — computes evaluation metrics on a given dataset split
- generate_shap_explanations — applies SHAP TreeExplainer to generate feature attributions
- main — orchestrates the full training pipeline for all three metals

get_feature_sets defines two feature column lists — the baseline set containing only technical and macroeconomic features, and the enhanced set additionally including all daily sentiment columns. generate_shap_explanations applies SHAP TreeExplainer to both models on the test set, saving global bar charts, beeswarm plots, and raw SHAP value arrays to disk.

Build_target_and_lag_features

Requirements

The build_target_and_lag_features function must construct the binary target variable using a forward shift to prevent look-ahead bias, and engineer lag features at t-1, t-2, and t-3 for six key variables to provide the model with short-term temporal context.

Design

```
RISK_THRESHOLD = -0.02

def build_target_and_lag_features(df: pd.DataFrame) -> pd.DataFrame:
  df = df.copy()
  df['target'] = (df['daily_return'].shift(-1) < RISK_THRESHOLD).astype(int)
  lag_cols = ['daily_return','log_return','volume_change',
        'rsi_14','macd_histogram','bollinger_width']
  for col in lag_cols:
    if col in df.columns:
      df[f'{col}_lag1'] = df[col].shift(1)
      df[f'{col}_lag2'] = df[col].shift(2)
      df[f'{col}_lag3'] = df[col].shift(3)
  # ... dropna on core columns, ffill remaining
  # (full implementation: scripts/04_model_training.py)
  return df
```

Testing

The function was validated by inspecting the target distribution for each metal shown in Figures 8, 9, and 10. Gold produced 49 risk events from 1,450 samples (3.4%), silver produced 147 from 1,386 (10.6%), and copper produced 112 from 1,452 (7.7%). The last row target value of 0 for all metals confirms the shift(-1) operation correctly drops the boundary row, ensuring no look-ahead bias.

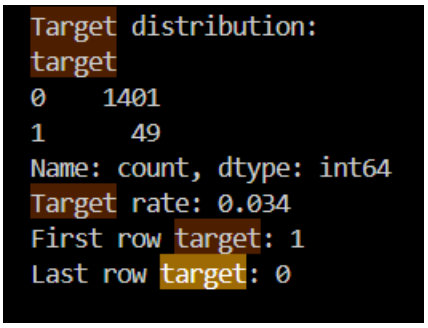


Figure 8: Terminal output confirming target variable construction for gold, showing 49 downside risk events from 1,450 total samples (3.4% positive class rate) and no look-ahead bias at the dataset boundary.

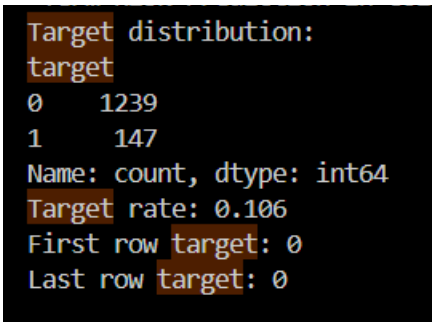


Figure 9: Terminal output confirming target variable construction for silver, showing 147 downside risk events from 1,386 total samples (10.6% positive class rate) and no look-ahead bias at the dataset boundary.

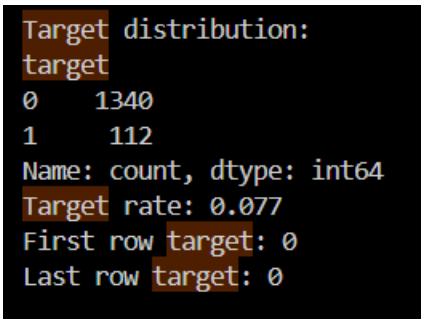


Figure 10: Terminal output confirming target variable construction for copper, showing 112 downside risk events from 1,452 total samples (7.7% positive class rate) and no look-ahead bias at the dataset boundary.

Temporal_split

Requirements

The temporal_split function must divide the dataset into training, validation, and test sets using a strictly chronological 60/20/20 split with no random shuffling, preserving temporal order to prevent look-ahead bias.

Design

```
FUNCTION temporal_split(df):
   Calculate total number of rows
   Set train_end = 60% of total rows
   Set val_end = 80% of total rows

   train = rows from start to train_end
   val = rows from train_end to val_end
   test = rows from val_end to end

   RETURN train, val, test
END FUNCTION
```

Implementation

```
TRAIN_RATIO   = 0.60
VAL_RATIO     = 0.20
..
def temporal_split(df: pd.DataFrame):
..
n = len(df)
   train_end = int(n * TRAIN_RATIO)
   val_end   = int(n * (TRAIN_RATIO + VAL_RATIO))

   train = df.iloc[:train_end].copy()
   val   = df.iloc[train_end:val_end].copy()
   test  = df.iloc[val_end:].copy()
..
   return train, val, test
-
```

Testing

The function was validated by inspecting the split sizes and date ranges printed during Script 04 execution, shown in Figures 11, 12, and 13. No overlap exists between any of the three sets and chronological ordering is preserved throughout.

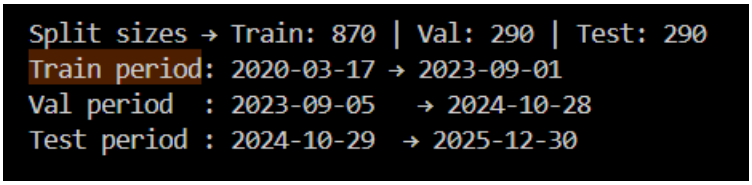


Figure 11: Gold Split

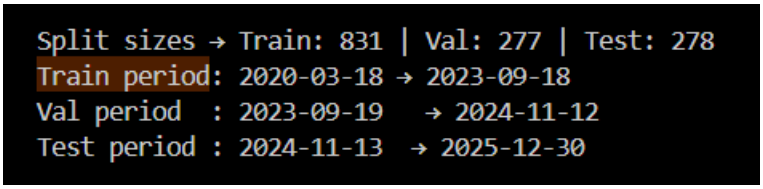


Figure 12: Silver Split

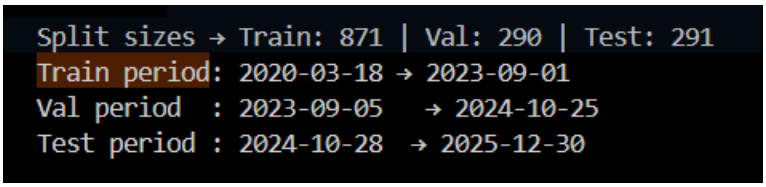


Figure 13: Copper Split

Train_random_forest

Requirements

The train_random_forest function must train a Random Forest classifier using balanced class weighting and GridSearchCV with TimeSeriesSplit cross-validation, returning the best estimator selected based on ROC-AUC performance on the validation folds.

Design

FUNCTION train_random_forest(X_train, y_train, label):
  Compute balanced class weights from training label distribution

  Define parameter grid:
    n_estimators: [100, 200, 300]
    max_depth: [5, 10, 15, None]
    min_samples_split: [2, 5, 10]
    min_samples_leaf: [1, 2, 4]
    max_features: [sqrt, log2]

```
  Create base RandomForestClassifier with class weights
  Create TimeSeriesSplit with 5 folds
  Run GridSearchCV scoring on ROC-AUC

  RETURN best estimator
END FUNCTION
```

Implementation

```
def train_random_forest(X_train, y_train, label):
  weights = compute_class_weight('balanced',
                    classes=np.unique(y_train), y=y_train)
  param_grid = {'n_estimators': [100,200,300],
          'max_depth': [5,10,15,None],
          'min_samples_split': [2,5,10],
          'min_samples_leaf': [1,2,4],
          'max_features': ['sqrt','log2']}
  grid_search = GridSearchCV(
    RandomForestClassifier(class_weight={0:weights[0],1:weights[1]},
                random_state=42, n_jobs=-1),
    param_grid, cv=TimeSeriesSplit(n_splits=5),
    scoring='roc_auc', n_jobs=-1, refit=True)
  grid_search.fit(X_train, y_train)
  return grid_search.best_estimator_
```

Testing

The function was validated by inspecting the GridSearchCV outputs shown in Figures 14, 15, and 16. Class weights confirm correct minority class upweighting — gold 16.7x, silver 4.4x, copper 6.1x. Different optimal configurations per metal confirm each asset has a distinct feature-outcome relationship.

```
    Class weights: {0: np.float64(0.5154028436018957), 1: np.float64(16.73076923076923)}
Fitting 5 folds for each of 216 candidates, totalling 1080 fits

    Best CV ROC-AUC : 0.7872
    Best params     : {'max_depth': 5, 'max_features': 'log2', 'min_samples_leaf': 4, 'min_samples_split': 2, 'n_estimators': 100}
```

Figure 14: GridSearchCV output for gold — best parameters and class weights confirming minority class upweighting of 16.7x

```
    Class weights: {0: np.float64(0.5637720488466758), 1: np.float64(4.42021276595744 7)}
Fitting 5 folds for each of 216 candidates, totalling 1080 fits

    Best CV ROC-AUC : 0.5531
    Best params     : {'max_depth': None, 'max_features': 'log2', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
```

Figure 15: GridSearchCV output for silver — best parameters and class weights confirming minority class upweighting of 4.4x

```
    Class weights: {0: np.float64(0.544375), 1: np.float64(6.133802816901408)}
Fitting 5 folds for each of 216 candidates, totalling 1080 fits

    Best CV ROC-AUC : 0.5515
    Best params     : {'max_depth': 5, 'max_features': 'log2', 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 200}
```

Figure 16: GridSearchCV output for copper — best parameters and class weights confirming minority class upweighting of 6.1x

4.4.2 Script 05: Model Evaluation

Script 05 loads the trained models saved by Script 04 and generates the full suite of evaluation outputs used in Chapter 5. The script reconstructs the identical feature matrix and test split used during training to ensure evaluation is performed on genuinely unseen data, computes all performance metrics for both baseline and enhanced model variants across all three metals, and produces visualisation plots and a summary CSV file. The script consists of the following functions:

- plot_roc_curve — generates ROC curves for baseline and enhanced models
- plot_pr_curve — generates precision-recall curves
- plot_calibration — generates probability calibration plots
- plot_confusion — generates confusion matrices at the 0.5 threshold
- main — orchestrates evaluation for all three metals and saves all outputs

The two most technically significant implementation decisions in Script 05 are the metric computation and the calibration curve strategy, documented below.

The first block shows how all six evaluation metrics are computed inline for each model variant.

-

```
for model, y_prob, y_pred, label in [
  (model_b, y_prob_b, y_pred_b, 'Baseline'),
  (model_e, y_prob_e, y_pred_e, 'Enhanced')
]:
  all_metrics.append({
    'metal':   metal_name,
    'model':   label,
    'roc_auc':  roc_auc_score(y_test, y_prob),
```

```
    'avg_prec':  average_precision_score(y_test, y_prob),
    'brier':     brier_score_loss(y_test, y_prob),
    'log_loss':  log_loss(y_test, y_prob),
    'f1':        f1_score(y_test, y_pred, zero_division=0),
    'precision': precision_score(y_test, y_pred, zero_division=0),
    'recall':    recall_score(y_test, y_pred, zero_division=0),
  })
```

-

ROC-AUC and Average Precision are computed from the continuous probability scores rather than binary predictions, making them threshold-independent measures of discriminative ability. F1-score, precision, and recall use zero_division=0 rather than the default which raises a warning, because with a 7.3% positive class rate the models rarely exceed the 0.5 threshold and produce all-zero predictions, making zero division a predictable and expected outcome rather than an error.

The second block shows the calibration curve implementation with the quantile binning strategy.

-

```
frac_b, mean_b = calibration_curve(
  y_true, y_prob_base,
  n_bins=10,
  strategy='quantile'
)
```

Testing

Script 05 was validated by running it end to end and confirming all outputs were generated successfully, as shown in Figure 17. The summary output confirms all six model variants were evaluated correctly.

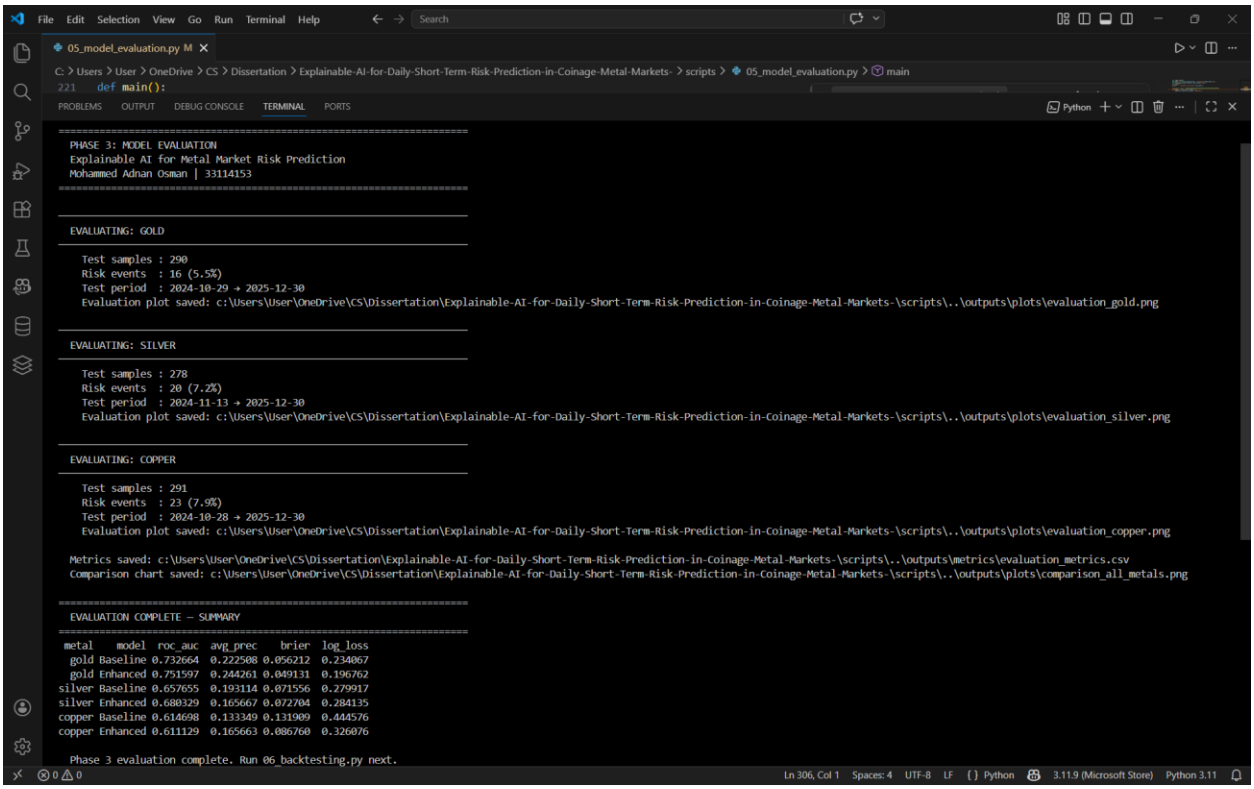


Figure 17: Terminal output confirming successful execution of Script 05, showing evaluation metrics for all six model variants across gold, silver, and copper — gold enhanced achieved the strongest ROC-AUC of 0.752 and Brier Score of 0.049.

4.4.3 Script 06: Back testing

Script 06 loads the trained models and runs walk-forward backtesting on unseen 2025 data, producing daily risk probability scores, timeline charts, SHAP waterfall plots, and prediction CSV files for each metal.

- get_engine — establishes the SQLAlchemy database connection
- load_full_dataset — loads and reconstructs the full feature matrix from the database
- walk_forward_predict — generates sequential daily risk predictions on the 2025 window
- plot_risk_timeline — plots daily risk probability over time with actual risk events marked

load_full_dataset reconstructs the identical feature matrix used during training by mirroring the SQL queries from Script 04. plot_risk_timeline produces a two-panel matplotlib figure showing baseline and enhanced model probabilities over time with actual risk events marked as red triangles.

walk_forward_predict

Requirements

The walk_forward_predict function must generate daily risk probability scores for each trading day in the 2025 backtest window using only features available on that day, returning a DataFrame containing the date, actual risk label, predicted probability, and binary prediction.

Design

```
FUNCTION walk_forward_predict(model, df_full, features, start_date, end_date):
  Filter df_full to backtest window
  IF empty: return empty DataFrame with warning

  Extract feature matrix for backtest window
  Generate risk probabilities using predict_proba
  Generate binary predictions using predict

  Construct results DataFrame with:
    date, daily_return, actual_risk,
    risk_prob, predicted, correct

  RETURN results DataFrame
END FUNCTION
```

Implementation

```
def walk_forward_predict(model, df_full, features,
                         start_date, end_date):
  backtest_df = df_full[
    (df_full['date'] >= start_date) &
    (df_full['date'] <= end_date)
  ].copy()
  if len(backtest_df) == 0:
    return pd.DataFrame()
  risk_probs  = model.predict_proba(backtest_df[features])[:, 1]
  predictions = model.predict(backtest_df[features])
  return pd.DataFrame({'date':       backtest_df['date'].values,
                       'actual_risk': backtest_df['target'].values,
                       'risk_prob':   risk_probs,
```

```
        'predicted':   predictions})
```

Testing

The function was validated through the Script 06 output shown in Figures 18 and 19. The gold enhanced model achieved a backtest ROC-AUC of 0.847, substantially higher than the 0.752 test set score, demonstrating strong generalisation to 2025 market conditions.

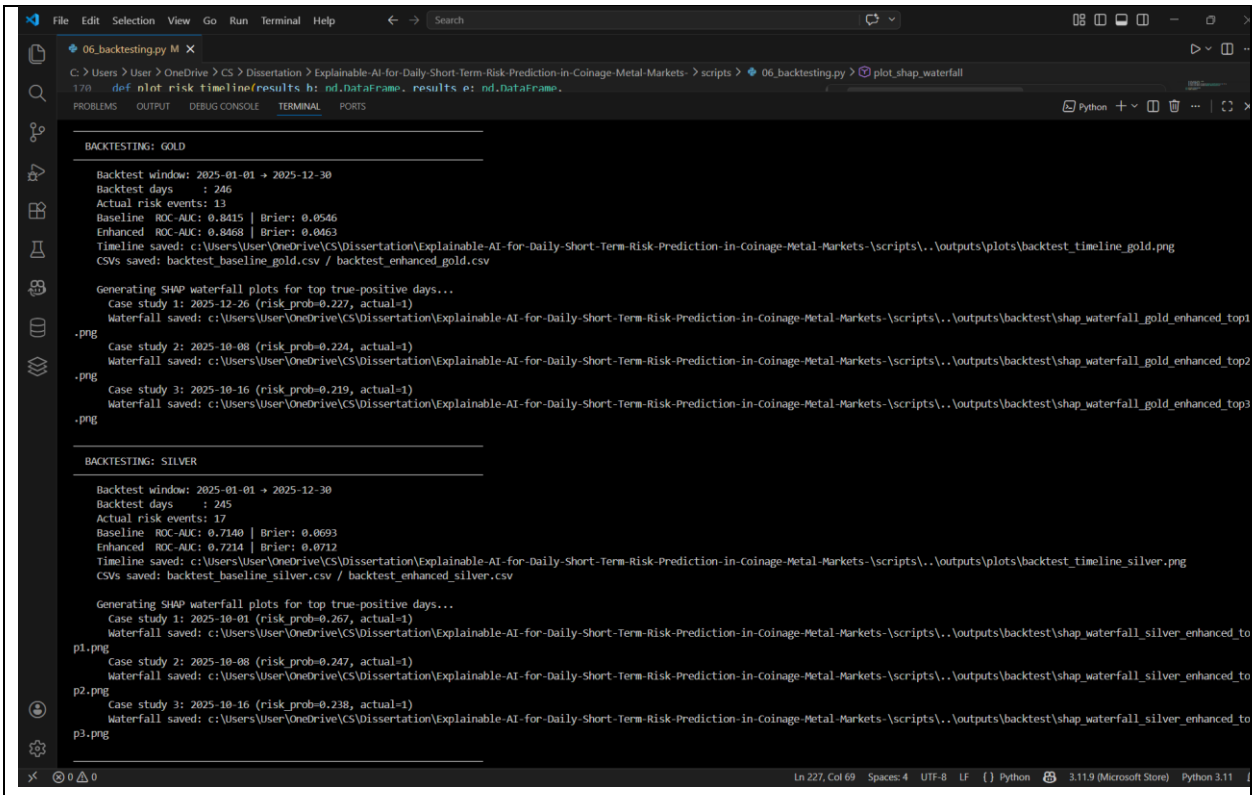


Figure 18: Terminal output confirming successful walk-forward backtesting for gold and silver on 2025 unseen data.

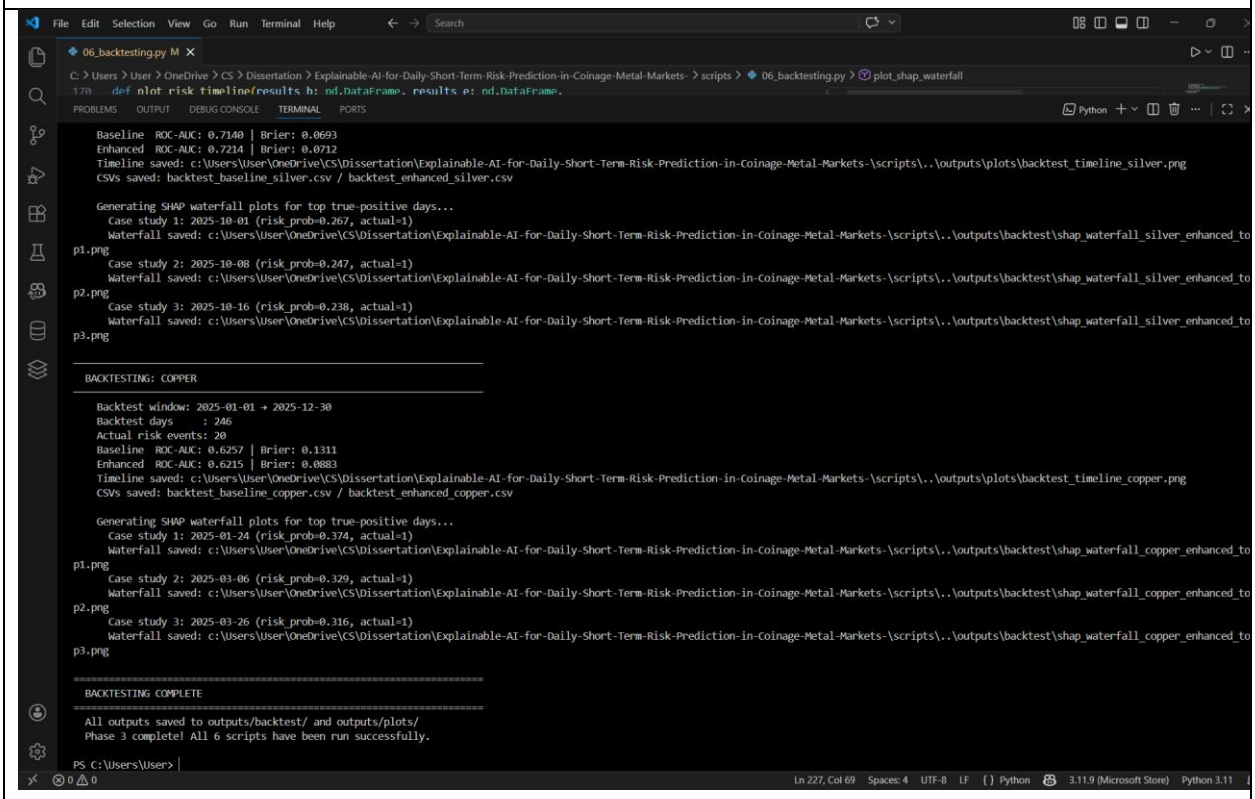


Figure 19: Terminal output confirming successful walk-forward backtesting for copper and BACKTESTING COMPLETE confirmation.

4.4.4 Script 07: SHAP Regeneration

Script 07 reloads the trained models and regenerates all SHAP bar charts and beeswarm plots using a corrected SHAP value extraction approach that handles array shape inconsistencies across SHAP library versions.

- get_engine — establishes the SQLAlchemy database connection
- load_test_data — loads and reconstructs the test split from the database
- get_shap_values — extracts class-1 SHAP values handling all possible output shapes
- make_shap_plots — generates corrected SHAP bar charts and beeswarm summary plots
- main — orchestrates SHAP regeneration for all three metals and both model variants

load_test_data reconstructs the identical test split used during training by mirroring the SQL queries and 60/20/20 split logic from Script 04.

Get_shap_values

Requirements

The get_shap_values function must extract class-1 SHAP values from the raw TreeExplainer output regardless of array shape, which varies between SHAP library versions, returning a consistent two-dimensional array of shape (n_samples, n_features).

Design

```
FUNCTION get_shap_values(model, X_test):
  Create TreeExplainer with tree_path_dependent perturbation
  Compute raw SHAP values
  Convert to numpy array

  IF shape is (2, n_samples, n_features):
    RETURN arr[1]  — second element is positive class
  IF shape is (n_samples, n_features, 2):
    RETURN arr[:, :, 1]  — last dimension index 1 is positive class
  IF shape is (n_samples, n_features):
    RETURN arr  — already single output
  ELSE:
    RETURN arr  — fallback
```

END FUNCTION

Implementation

```
def get_shap_values(model, X_test):
  explainer = shap.TreeExplainer(
    model,
    feature_perturbation='tree_path_dependent'
  )
  raw = explainer.shap_values(X_test)
  arr = np.array(raw)

  if arr.ndim == 3 and arr.shape[0] == 2:
    return arr[1]

  if arr.ndim == 3 and arr.shape[2] == 2:
    return arr[:, :, 1]

  if arr.ndim == 2:
    return arr

  return arr
```

Testing

The function was developed in response to a SHAP API inconsistency where raw output shape varies between library versions. The three-branch conditional ensures correct positive class values are extracted regardless of which SHAP version is installed, making the pipeline robust across different deployment environments.

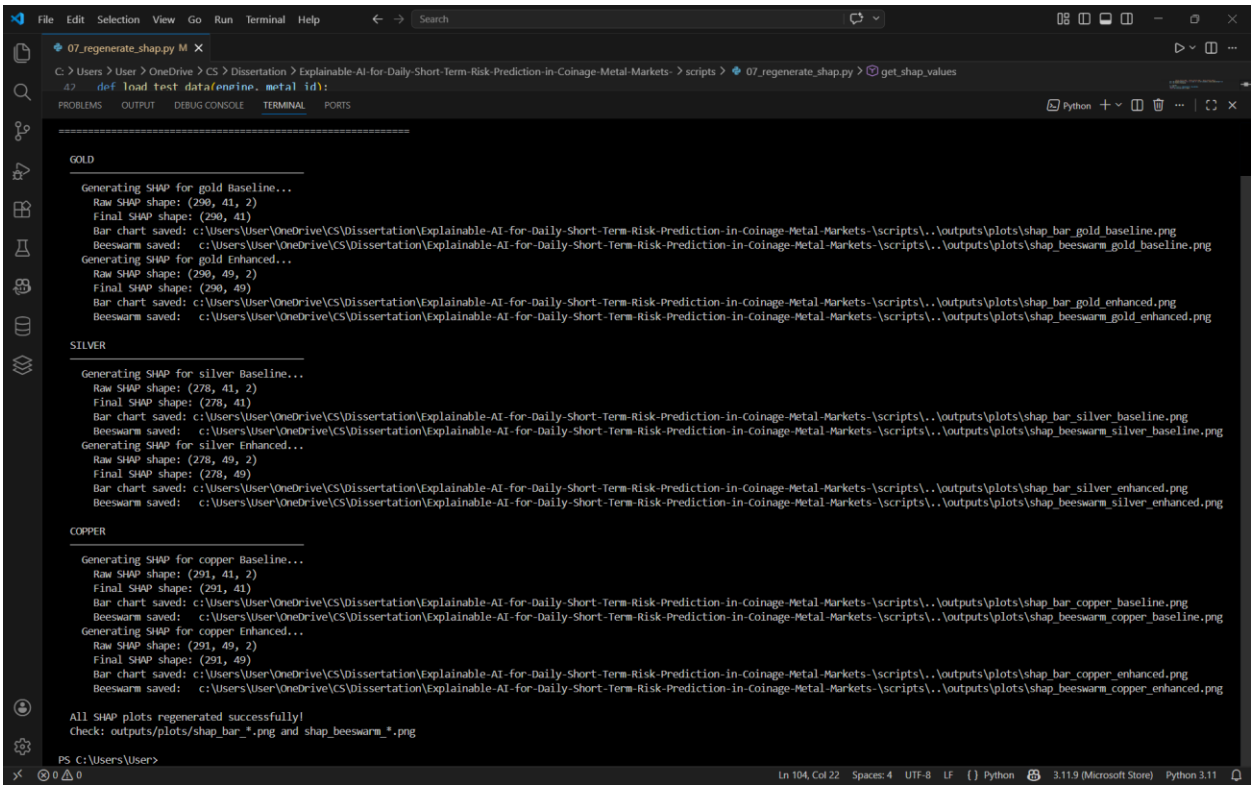


Figure 20: Terminal output confirming successful SHAP plot regeneration for all three metals and both model variants via Script 07.

4.5 Stream-lit Dashboard

The system includes an interactive dashboard built with Streamlit, providing an interface to explore model outputs, SHAP explanations, and live risk predictions. This dashboard meets the real-time display requirement and serves as the primary demonstration tool for the oral presentation.

The dashboard features four pages. The Live Prediction page fetches market data via yFinance, engineers 49 features, and runs the enhanced model to show colour-coded risk probabilities and real-time SHAP charts. The Model Performance page compares test set metrics for the baseline and enhanced models. The SHAP Analysis page displays the bar charts and plots from Script 07, while the Backtesting page shows the ROC-AUC scores and timeline plots from Script 06.

Testing and Verification

On 27 April 2026, the dashboard was tested with live market data. Figure 40 shows the **Live Prediction** page generating risk scores for gold ($4,717.10), silver ($75.56), and copper ($6.08), with all metals classified as "low risk" (4%, 14%, and 19% probability). The SHAP charts confirmed that treasury_yield_10y was the main driver for gold, matching the global

analysis in Chapter 5. Figure 41 shows the **Model Performance** page correctly displaying gold metrics (ROC-AUC 0.7516, Brier Score 0.0491, and Log Loss 0.1968), which are consistent with the results from Script 05.

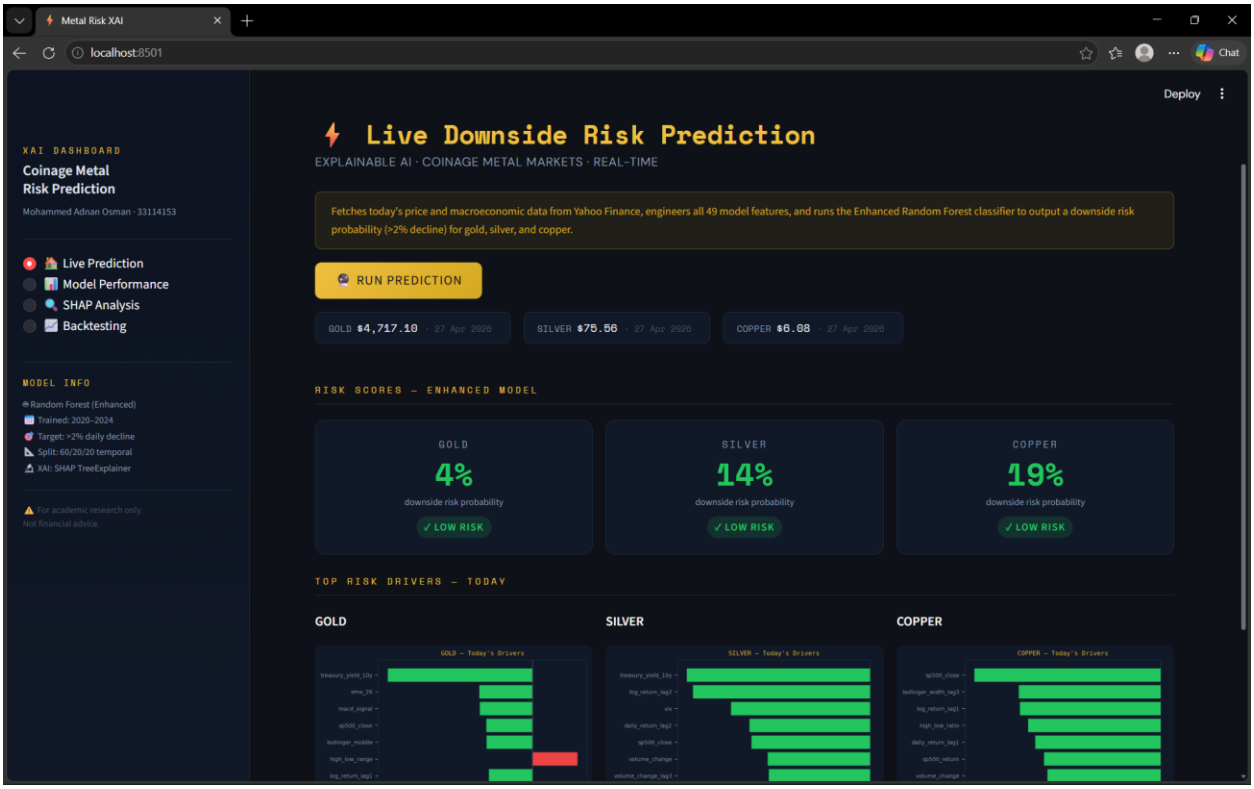
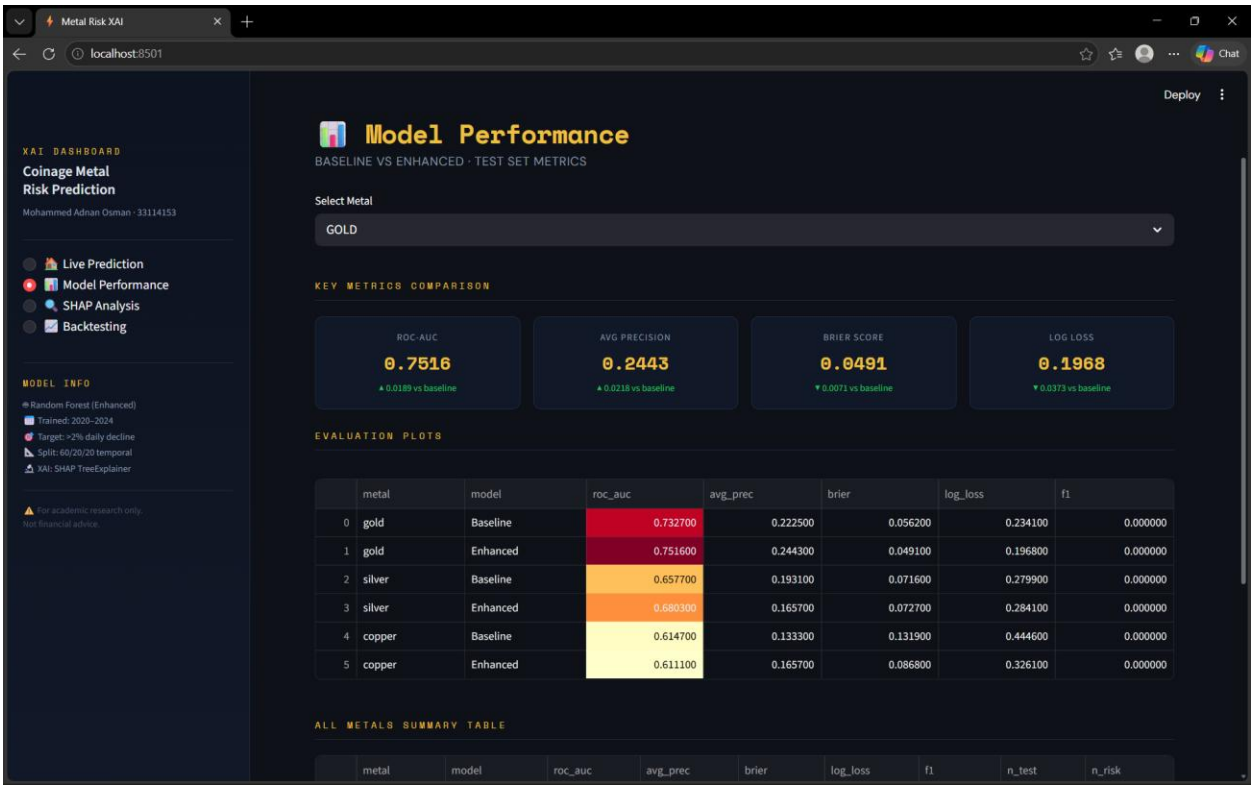


Figure 40: *Live Prediction page showing real-time downside risk scores for gold (4%), silver (14%), and copper (19%) on 27 April 2026, with live SHAP driver charts for each metal.*



	metal	model	roc_auc	avg_prec	brier	log_loss	f1
0	gold	Baseline	0.732700	0.222500	0.056200	0.234100	0.000000
1	gold	Enhanced	0.751600	0.244300	0.049100	0.196800	0.000000
2	silver	Baseline	0.657700	0.193100	0.071600	0.279900	0.000000
3	silver	Enhanced	0.680300	0.165700	0.072700	0.284100	0.000000
4	copper	Baseline	0.614700	0.133300	0.131900	0.444600	0.000000
5	copper	Enhanced	0.611100	0.165700	0.086800	0.326100	0.000000

Figure 41: *Model Performance page displaying gold enhanced model metrics — ROC-AUC 0.7516, Brier Score 0.0491, Log Loss 0.1968 — with baseline comparison deltas and full six-model summary table.*

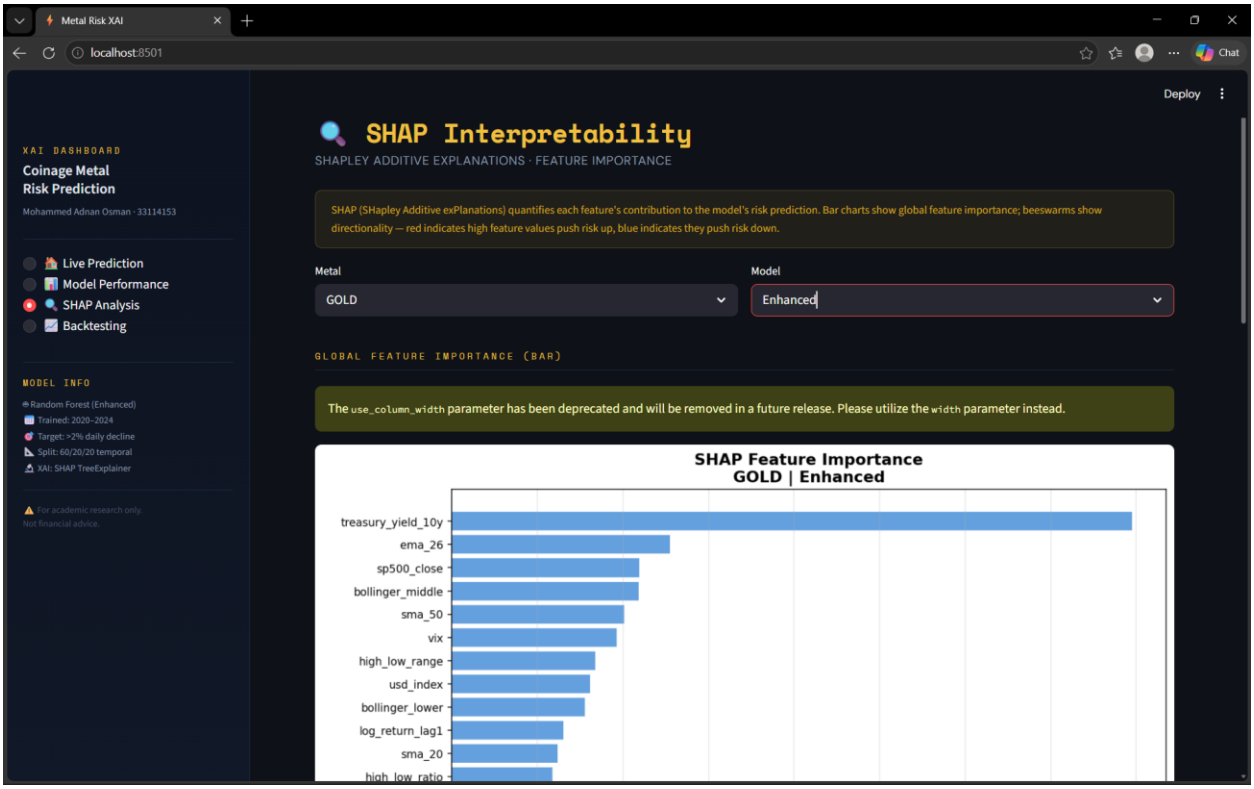


Figure 42: *SHAP Interpretability page displaying the global feature importance bar chart for the gold baseline model, confirming treasury_yield_10y as the dominant risk driver.*

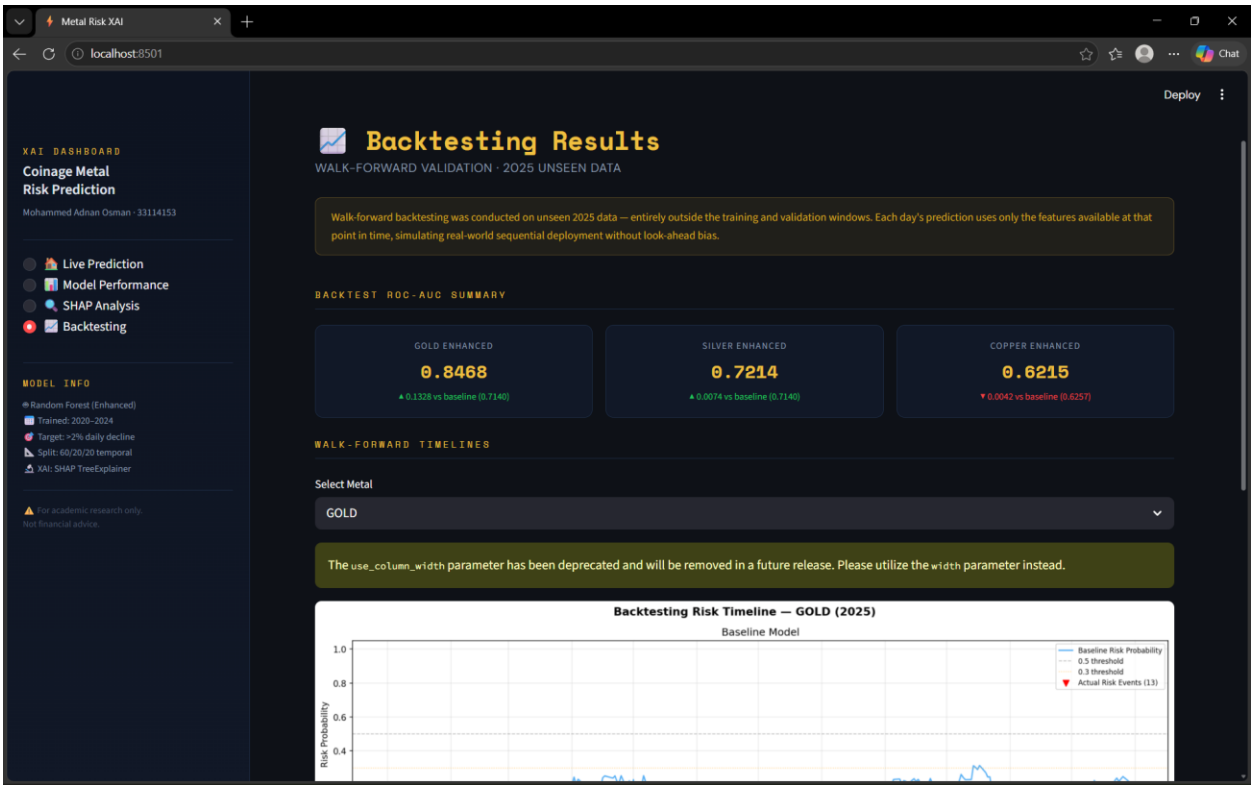


Figure 43: *Backtesting Results page displaying walk-forward ROC-AUC scores of 0.8468 for gold, 0.7214 for silver, and 0.6215 for copper enhanced models, with the gold backtesting timeline plot.*

4.6 System Requirements

Table 2: System requirements summary

Requirement	Specification
Programming Language	Python 3.11
Database	PostgreSQL 15+ (local instance via pgAdmin)
ML Framework	Scikit-learn 1.3+
Explainability	Shap 0.44+
NLP Model	FinBERT via Hugging Face transformers library
Data Source (Price)	yFinance (Yahoo Finance API)
Data Source (News)	NewsAPI (financial news headlines)
Version Control	Git / GitHub
Credential Management	.env file via python-doteny

5 Results and Analysis

5.1 Dataset Overview and Composition

The dataset includes 4,525 trading day records across three metals from January 2020 to December 2024. Price and macroeconomic data were collected via yFinance and stored in a PostgreSQL database, while technical indicators were computed using a sliding window approach. There are 331 downside risk events (next-day price declines exceeding two percent), representing 7.3% of the data. This frequency confirms that the use of balanced class weighting was necessary to handle the class imbalance.

Sentiment features from FinBERT-processed news were available for only 31 trading days due to NewsAPI free tier limits. For the remaining days, missing sentiment values were filled using forward-fill imputation. This sparsity is a significant limitation, which is explored in later chapters, as it limits the performance gains expected from sentiment integration.

5.2 Model Performance on Test Set

Table 3 presents the performance of the baseline and enhanced Random Forest models across all three metals on the held-out test set, evaluated using ROC-AUC, Brier Score, Log Loss, and F1-score.

Table 3: Test Set Performance: Baseline vs Enhanced Models

Metal	Baseline ROC-AUC	Enhanced ROC-AUC	Baseline Brier	Enhanced Brier	Baseline AP	Enhanced AP
Gold	0.733	0.752	0.056	0.049	0.223	0.244
Silver	0.658	0.680	0.072	0.073	0.193	0.166
Copper	0.615	0.611	0.132	0.087	0.133	0.166

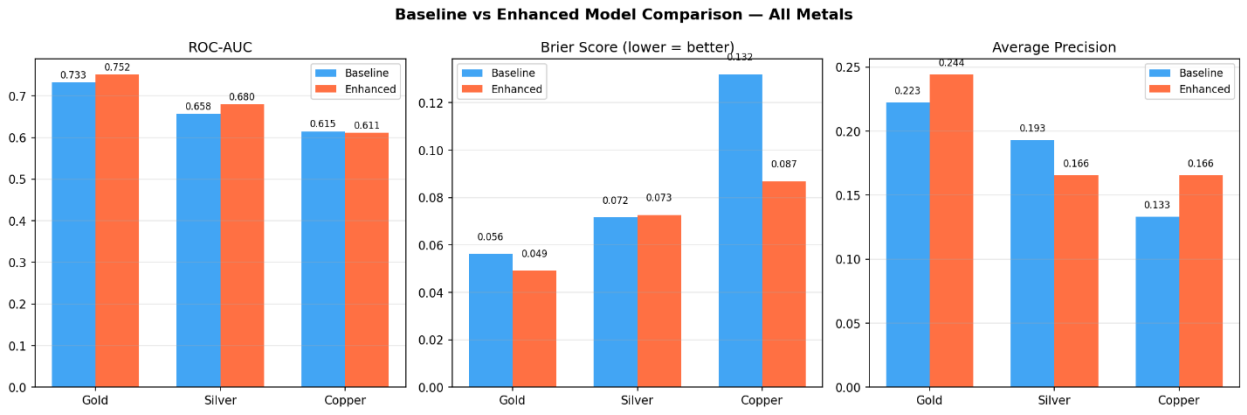


Figure 21: Baseline vs enhanced model comparison across all three metals: ROC-AUC, Brier Score, and Average Precision

The gold enhanced model achieved the most consistent improvement across metrics. ROC-AUC increased from 0.733 to 0.752, Brier Score improved from 0.056 to 0.049, and Average Precision increased from 0.223 to 0.244. These improvements across all three measures indicate that sentiment integration provided meaningful incremental value for gold, improving both the model's ability to rank risk days correctly and the accuracy of its predicted risk probabilities. The individual gold evaluation panel is presented in Figure 22.

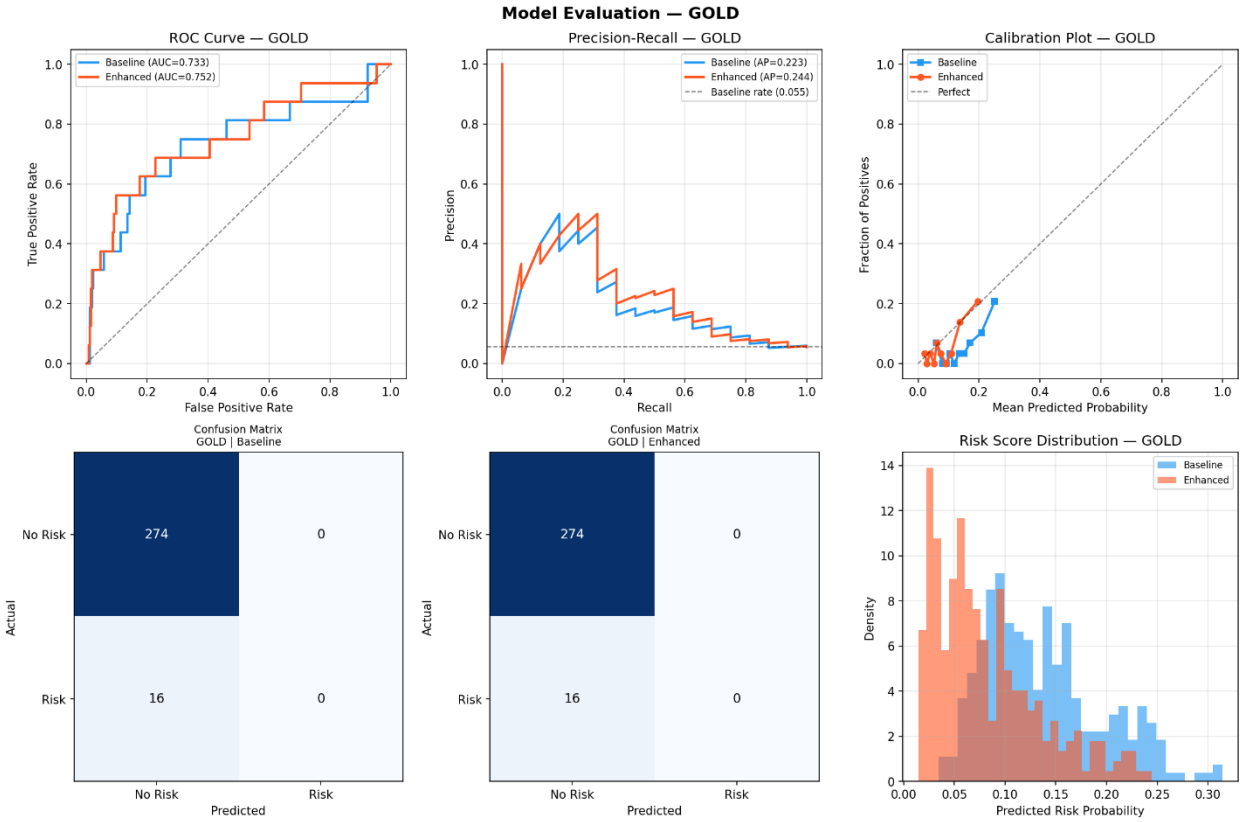


Figure 22: Model evaluation outputs for gold: ROC curve, precision-recall curve, calibration plot, confusion matrices, and risk score distribution (baseline vs enhanced)

Silver followed a mixed pattern, as shown in Figure 23. The ROC-AUC improved from 0.658 to 0.680 with sentimental features, and the Brier Score showed negligible change. However, Average Precision declined from 0.193 to 0.166, indicating that while the enhanced model ranks risk days more accurately across all thresholds, its precision at high recall levels worsened.

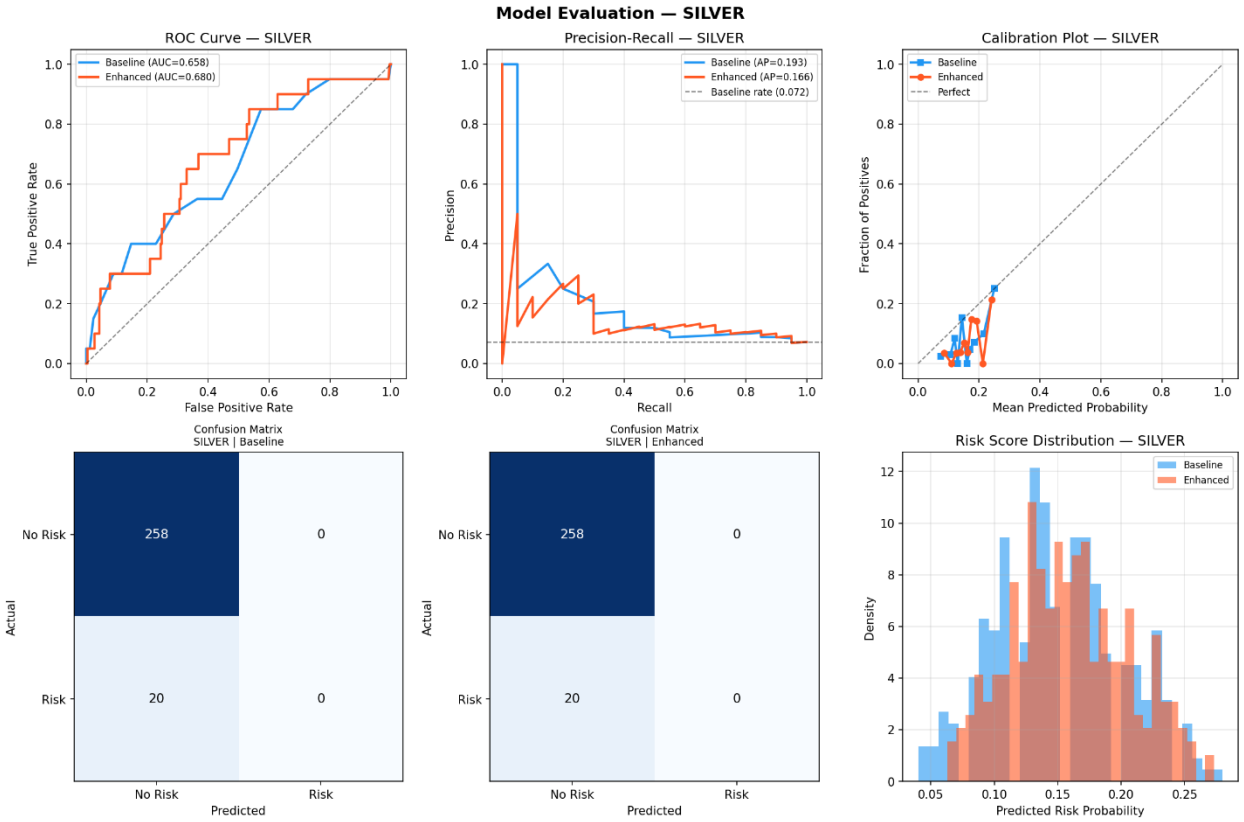


Figure 23: Model evaluation outputs for silver: ROC curve, precision-recall curve, calibration plot, confusion matrices, and risk score distribution (baseline vs enhanced)

Copper's results (Figure 24) show a minor ROC-AUC decrease from 0.615 to 0.611. However, the Brier Score improved significantly from 0.132 to 0.087, and Average Precision rose from 0.133 to 0.166. The risk score distribution is particularly revealing. The baseline model spreads probabilities across a wide range of 0.1 to 0.5, whereas the enhanced model produces more concentrated, lower-risk predictions between 0.1 and 0.35. This compression explains the Brier Score improvement because the enhanced model is less likely to assign high-risk probabilities on non-event days. Meanwhile, the stable ROC-AUC indicates that the model's ability to rank risk days remains unchanged.

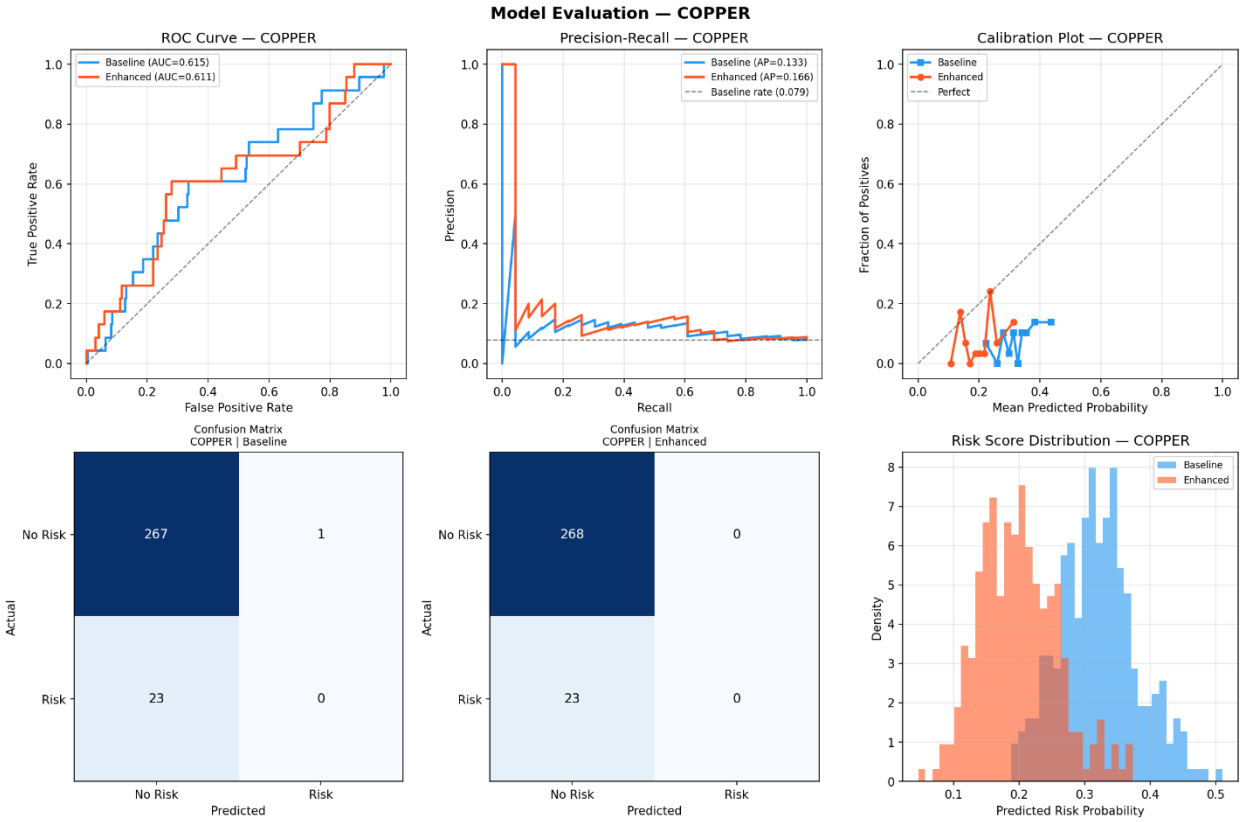


Figure 24: Model evaluation outputs for copper: ROC curve, precision-recall curve, calibration plot, confusion matrices, and risk score distribution (baseline vs enhanced)

5.3 SHAP Feature Importance Analyis

SHAP TreeExplainer was applied to both baseline and enhanced models across all three metals, generating global feature importance bar charts and beeswarm summary plots on the test set. Figures 25 through 36 present these outputs for all metals and model variants.

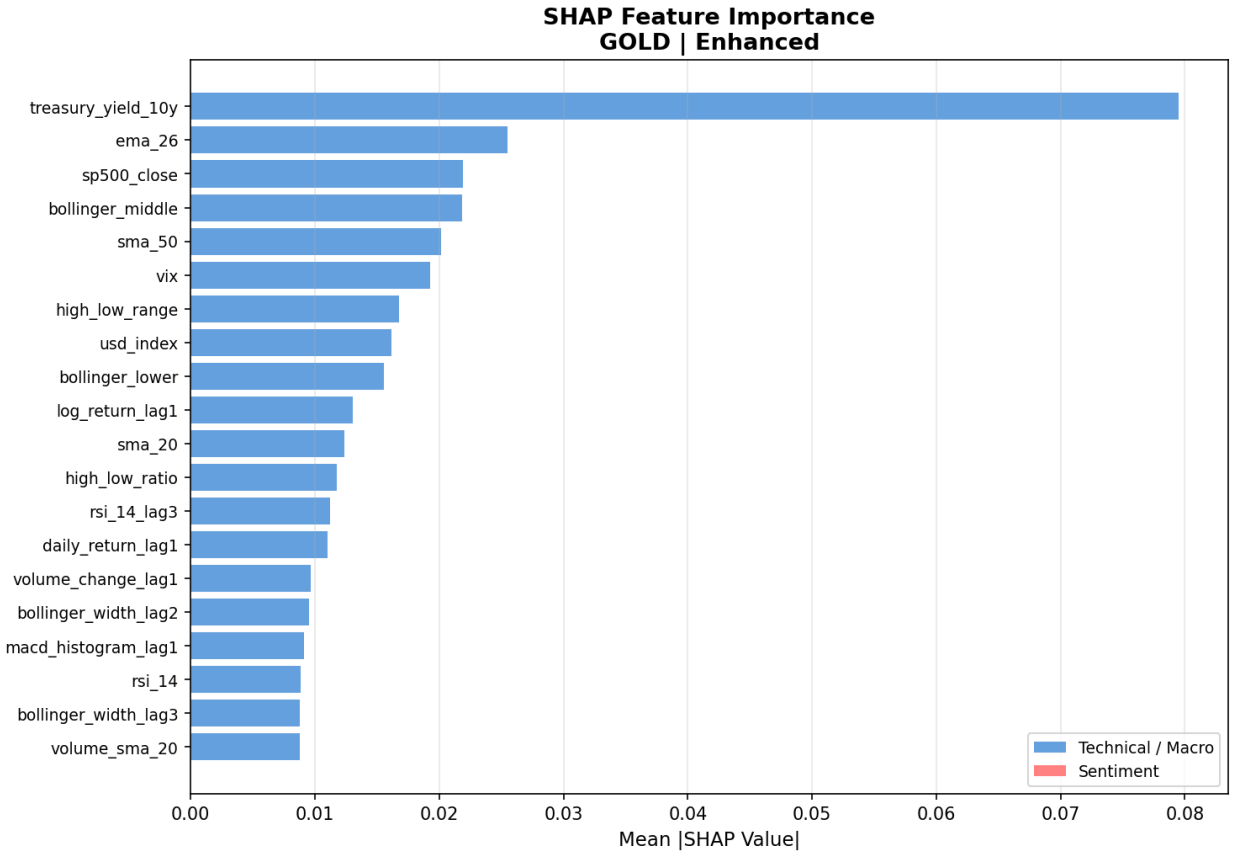


Figure 25: SHAP feature importance bar chart: Gold Baseline model. Mean absolute SHAP values across the test set.

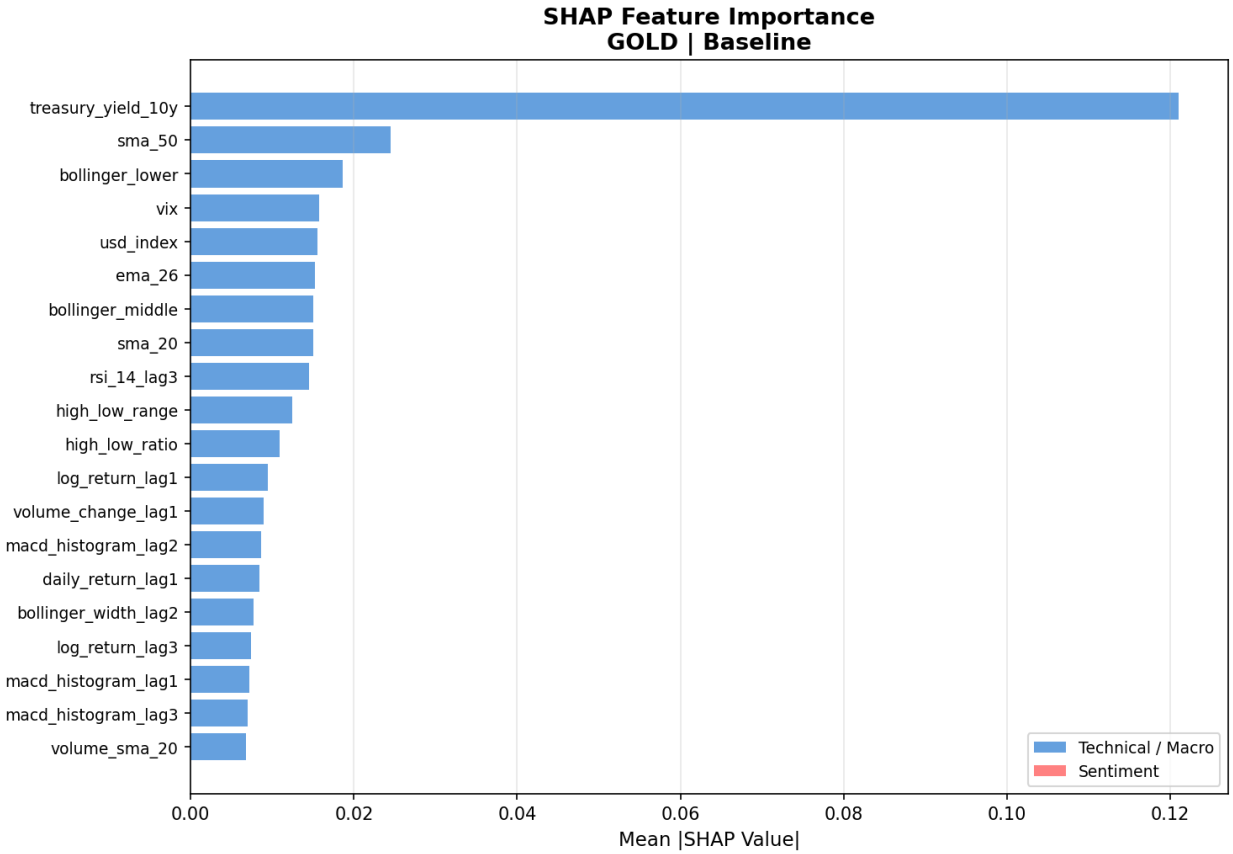


Figure 26: SHAP feature importance bar chart: Gold Baseline model. Mean absolute SHAP values across the test set.

As shown in the analysis, the 10-year US Treasury yield (treasury_yield_10y) is the dominant predictive feature for gold in both models. In the baseline model, it has a mean absolute SHAP value of 0.12, which is substantially higher than the second-ranked variable (sma_50) at 0.025. This dominance continues in the enhanced model where the yield retains the top ranking with a SHAP value of 0.08. Its reduced importance in the enhanced model reflects how predictive weight is redistributed across the broader feature set when sentiment variables are added rather than any change in the underlying predictive relationship. The corresponding beeswarm plots are presented later in this chapter.

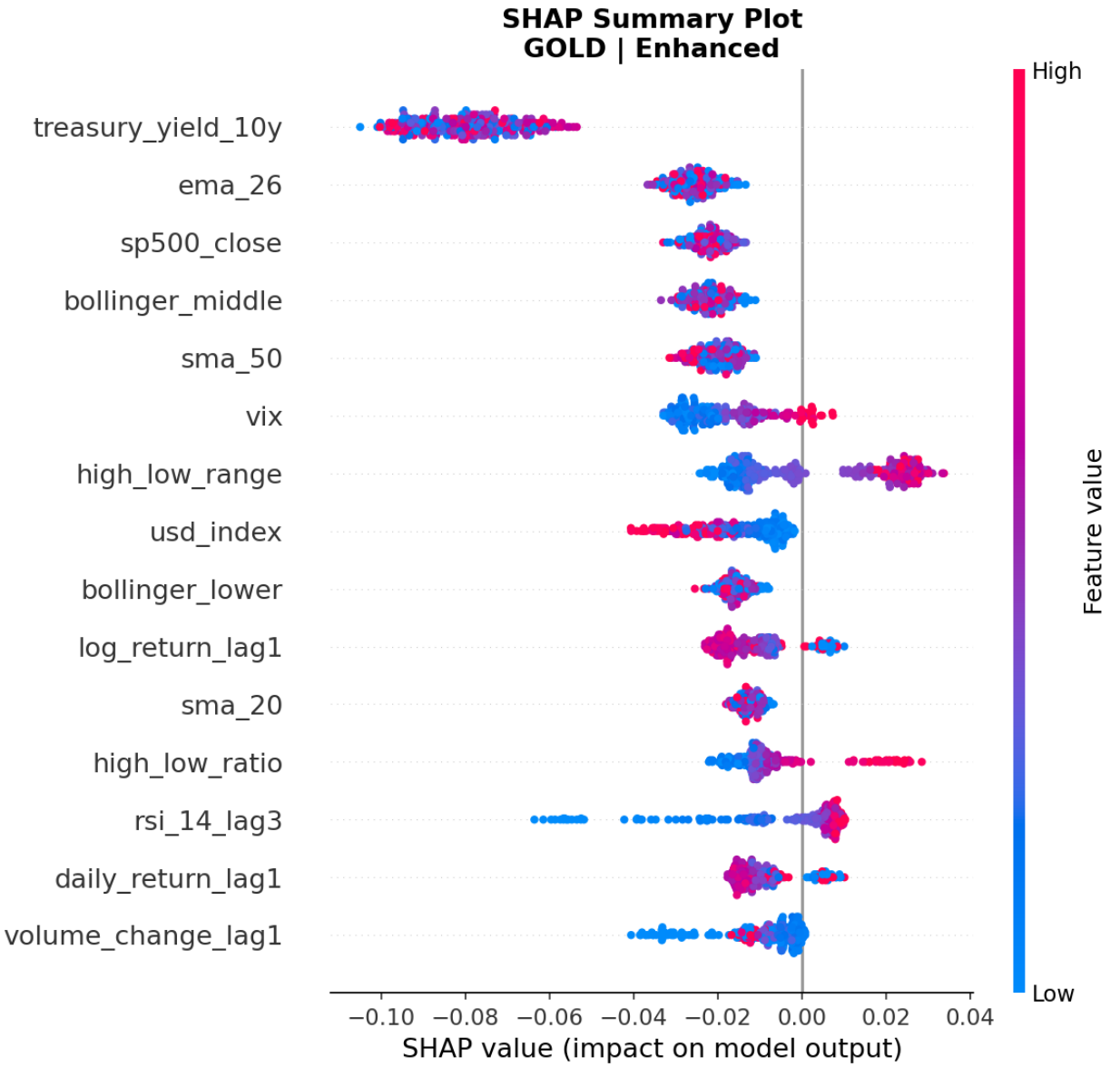


Figure 27: SHAP beeswarm summary plot: Gold Enhanced model.

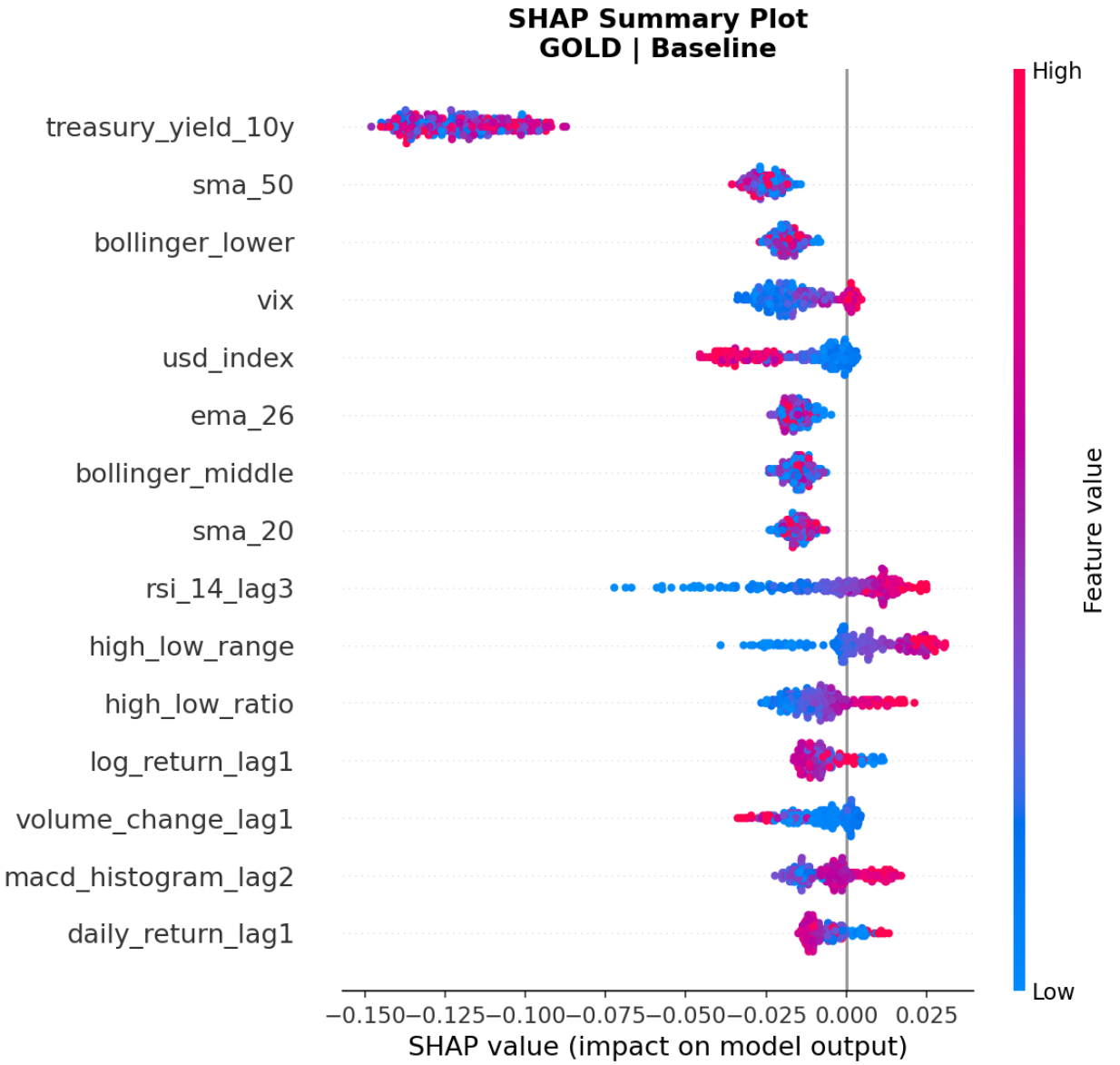


Figure 28: SHAP beeswarm summary plot: Gold Baseline model. Each point represents one test observation; colour indicates feature value (red = high, blue = low).

The beeswarm plot for the gold baseline provides directional insight into how the Treasury yield influences predictions. Most data points for the 10-year Treasury yield are clustered to the left of zero. This indicates that elevated Treasury yields are generally associated with a lower predicted downside risk in the model. The US Dollar Index shows a contrasting pattern, as high values are distributed to the right of zero. This is consistent with the expectation that dollar strength increases predicted downside risk for gold. The VIX feature shows a dispersed pattern around zero, which suggests a complex, non-linear contribution that changes depending on market conditions.

For silver, the VIX is the dominant feature, followed by the 10-year Treasury yield and the S&P 500 return. Volume-related features also rank more prominently for silver than they do for gold.

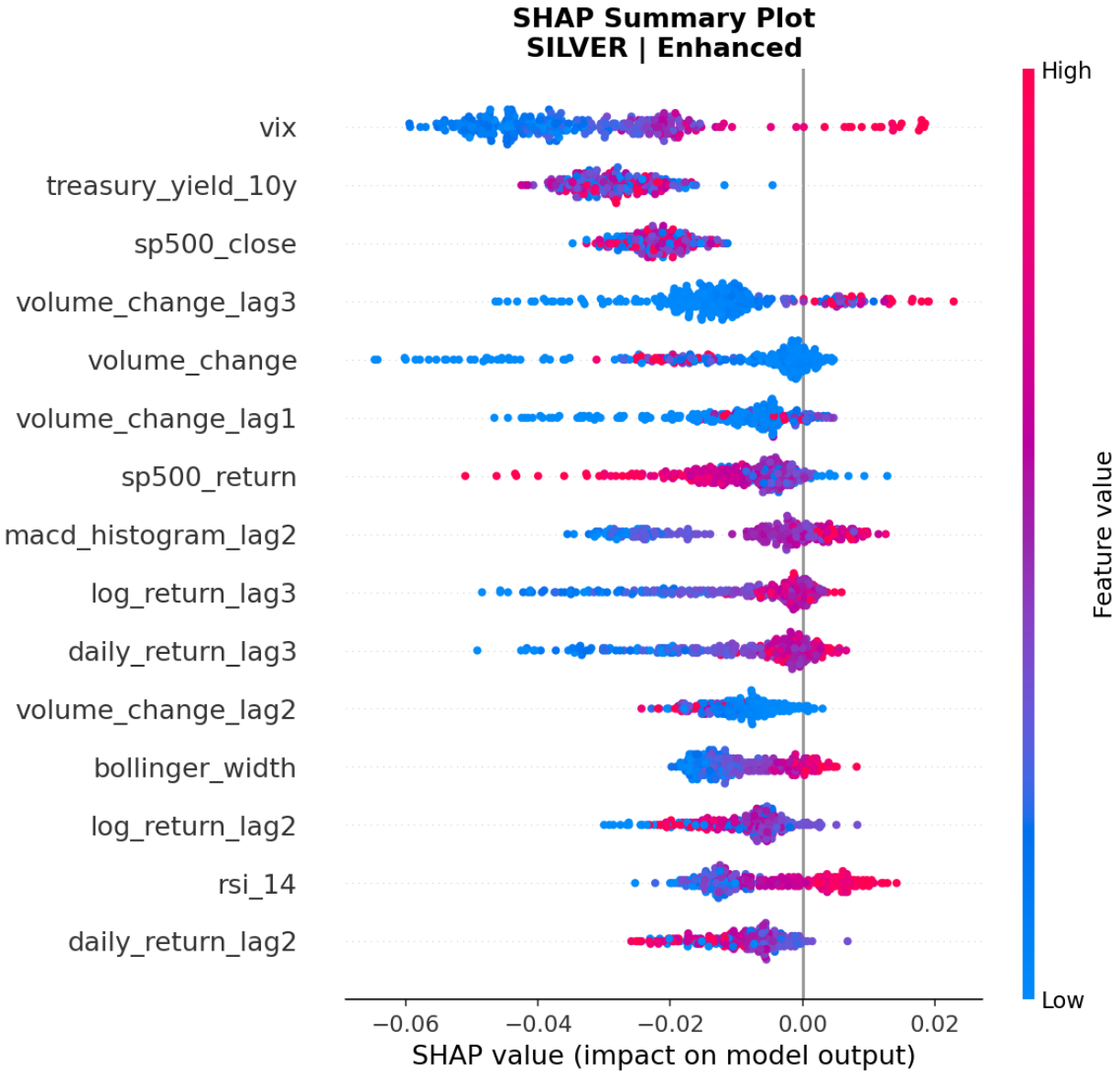


Figure 29: SHAP beeswarm summary plot: Silver Enhanced model.

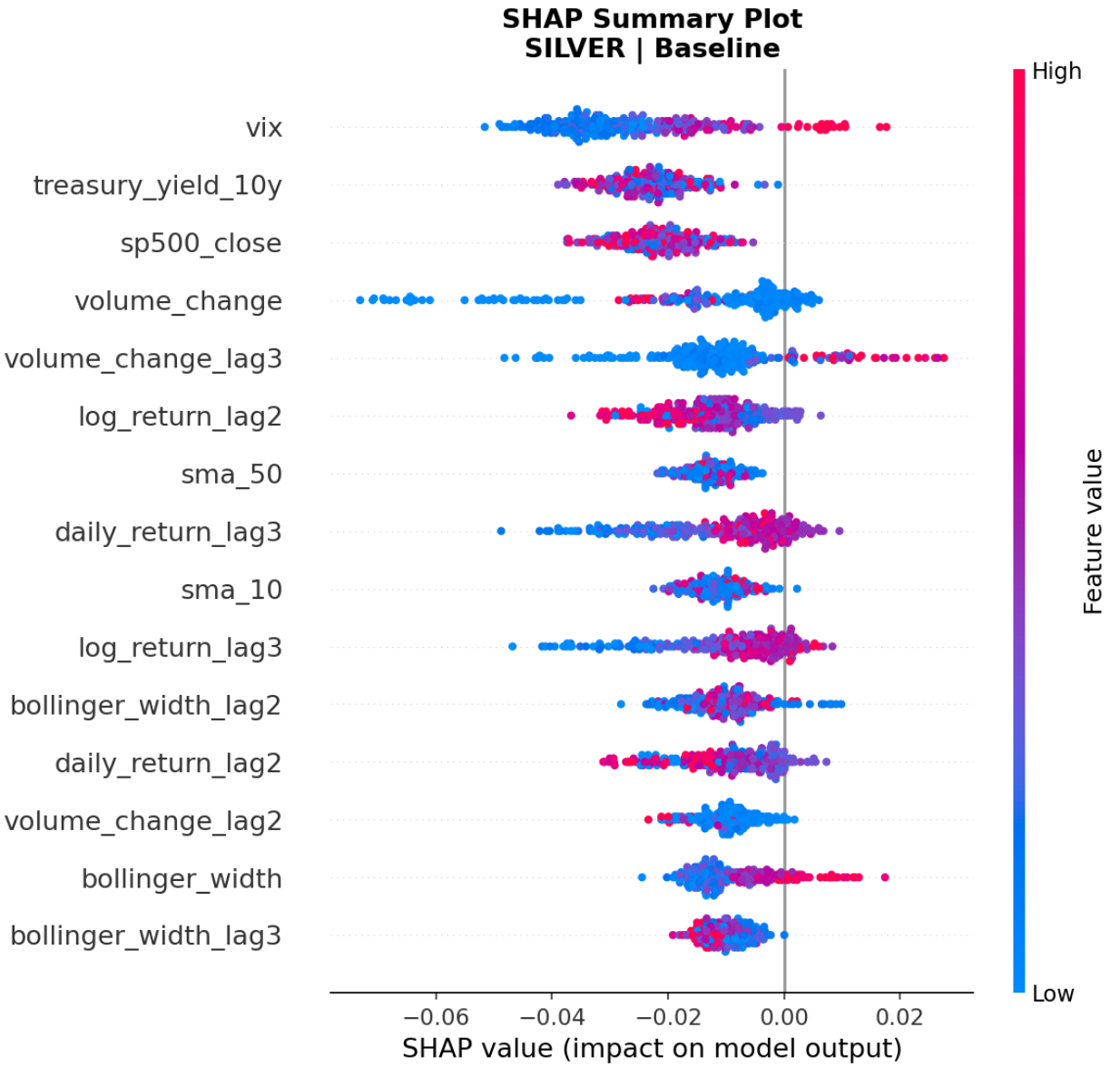


Figure 30: SHAP beeswarm summary plot: Silver Baseline model.

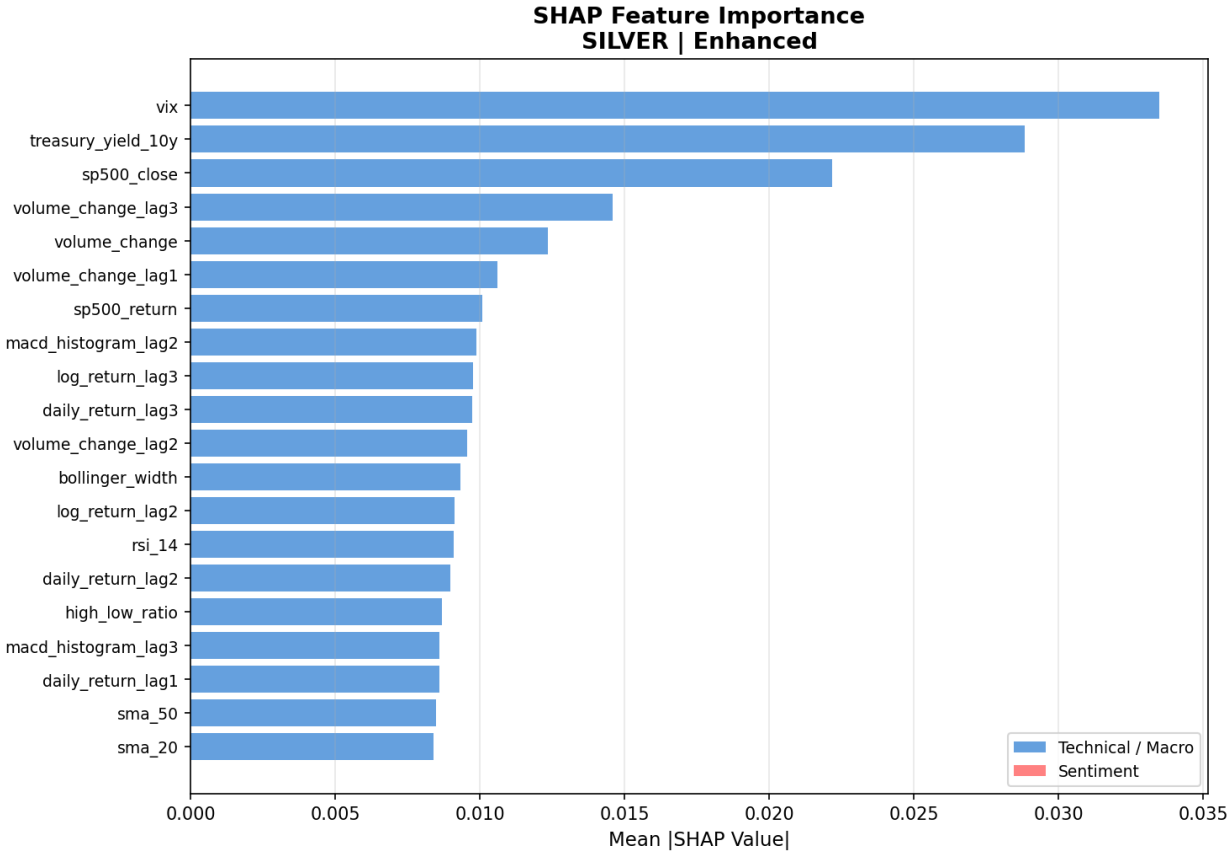


Figure 31: SHAP feature importance bar chart: Silver Enhanced model.

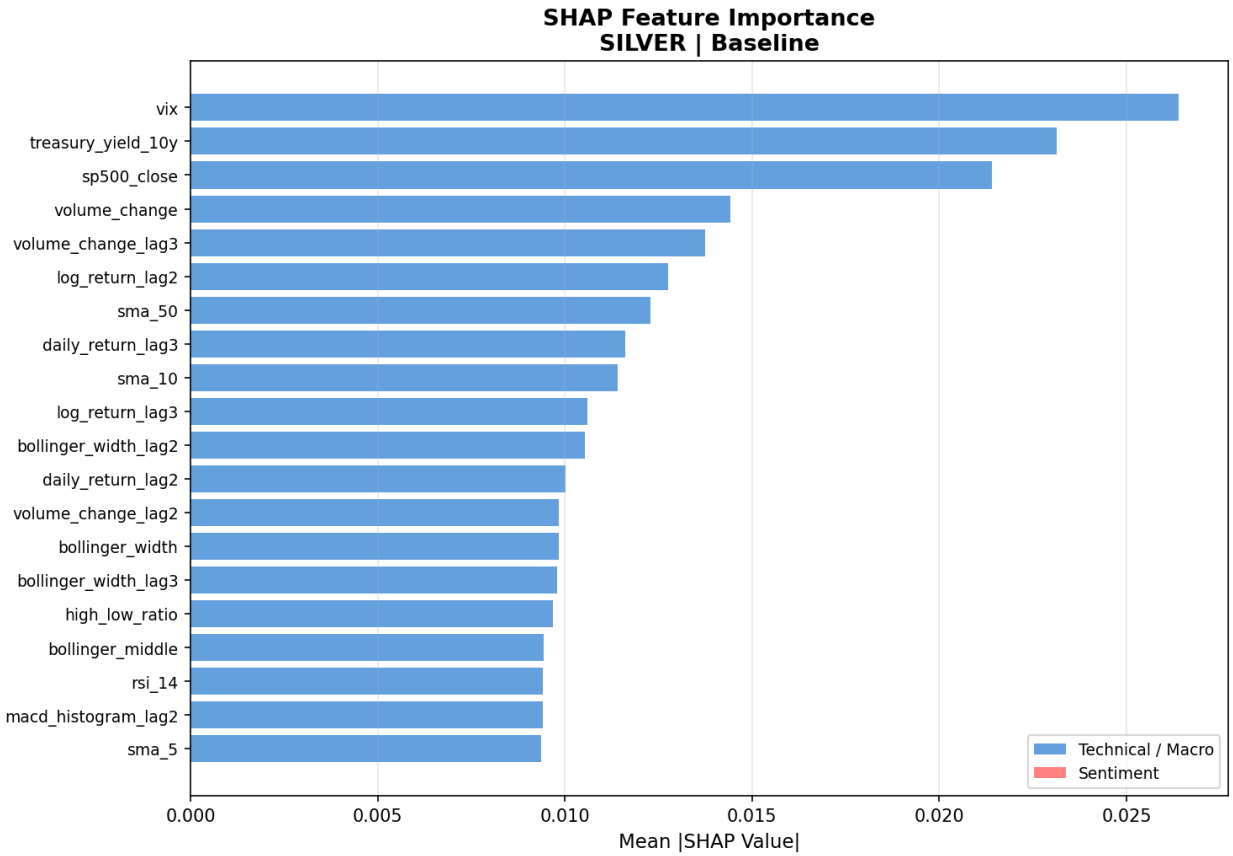


Figure 32: SHAP feature importance bar chart: Silver Baseline model.

Copper's SHAP profile diverges notably from the precious metals. The S&P 500 return is the dominant baseline feature at 0.033, followed by the S&P 500 close at 0.024, the three-day volume change, and the high-low range. It makes sense that equity market returns are the top predictor for copper because copper is a global growth barometer. Broad equity market movements reflect the same macroeconomic sentiment that drives copper demand. The beeswarm plots confirm that high S&P 500 returns are associated with negative SHAP values. This means strong equity returns reduce predicted copper downside risk, which is consistent with the procyclical nature of copper.

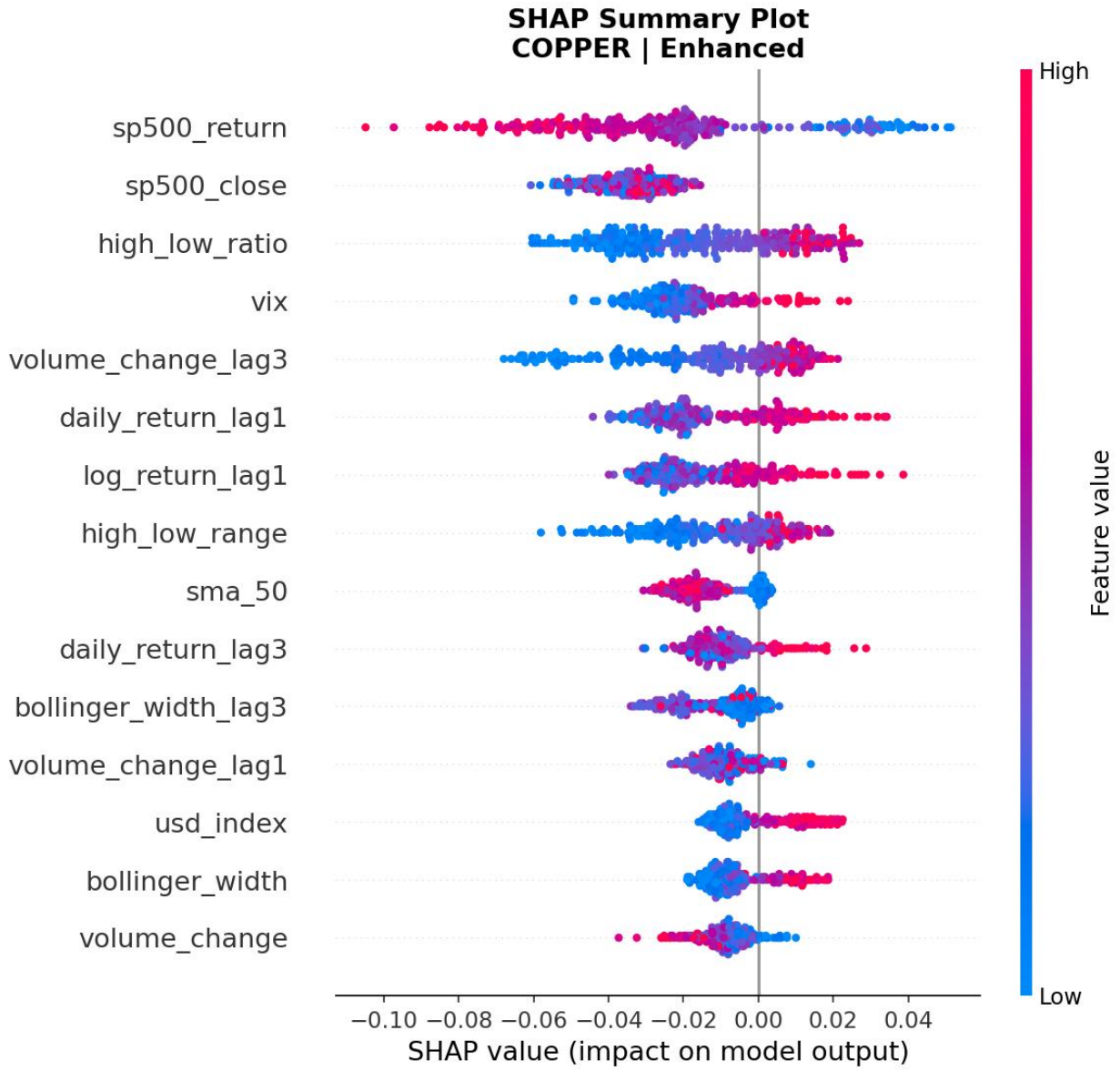


Figure 33: SHAP beeswarm summary plot: Copper Enhanced model.

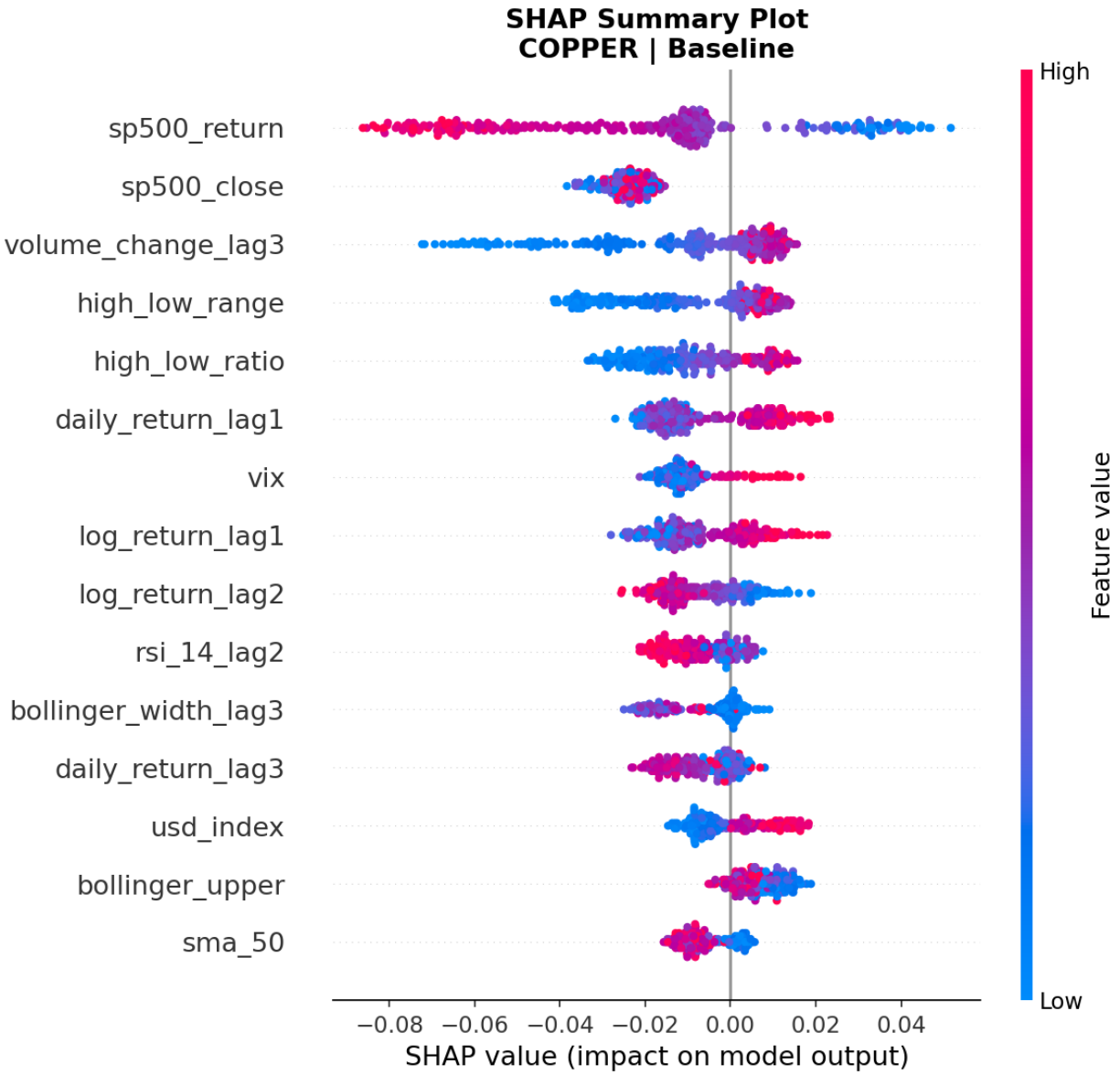


Figure 34: SHAP beeswarm summary plot: Copper Baseline model.

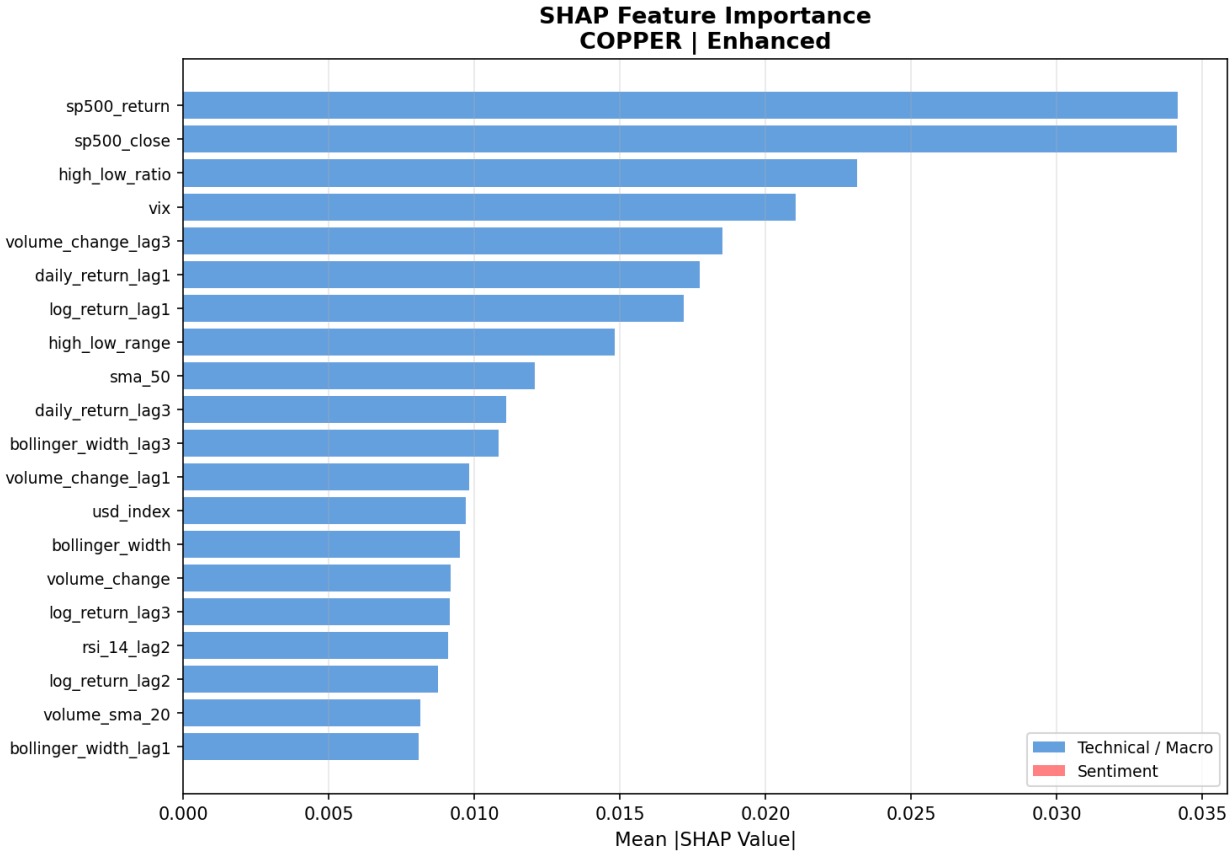


Figure 35: SHAP feature importance bar chart: Copper Enhanced model.

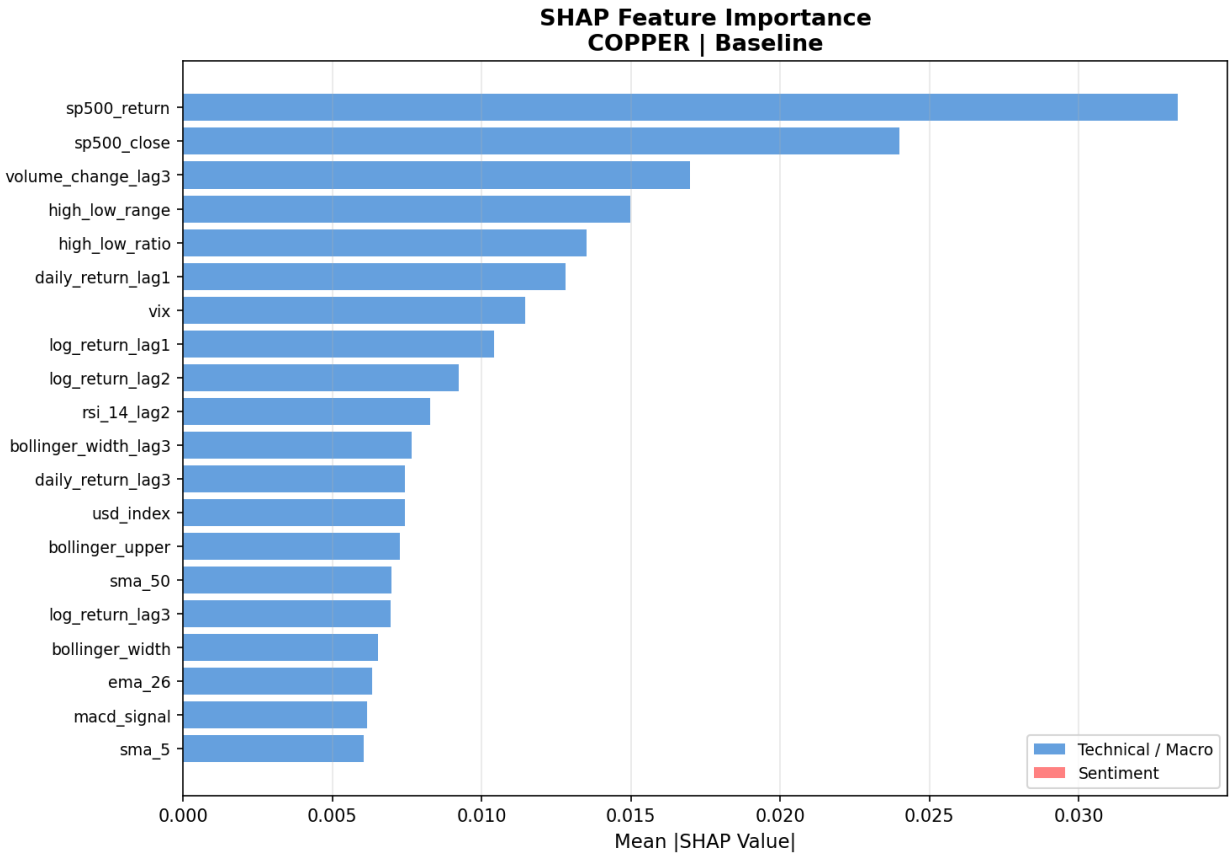


Figure 36: SHAP feature importance bar chart: Copper Baseline model.

5.4 Sentiment Feature Impact

For gold, sentiment integration produced consistent improvements. ROC-AUC increased from 0.733 to 0.752, the Brier Score improved from 0.056 to 0.049, and Average Precision increased from 0.223 to 0.244. This was the only metal where sentiment produced gains across every metric. This suggests that for gold, where prices are driven by macroeconomic and geopolitical news, FinBERT headlines provide a useful signal that complements price and macro data.

Silver's results were more mixed. ROC-AUC improved from 0.658 to 0.680, but Average Precision declined from 0.193 to 0.166. This indicates the enhanced model ranks risk days more accurately overall, but with lower precision when identifying the most critical risk periods. Silver's dual role as both a safe-haven asset and an industrial metal may cause this, as sentiment signals regarding precious metal demand may conflict with industrial demand signals.

Copper's outcome was the most distinctive. ROC-AUC showed a minor decline of 0.004, confirming no meaningful improvement in discriminative ability. However, the Brier Score

improved substantially from 0.132 to 0.087, which is the largest calibration gain in the study. This suggests sentiment acts as a regularising influence for copper, as it reduces incorrectly high-risk probabilities on non-event days without improving the model's fundamental ability to rank risk days.

5.5 Back testing on Unseen 2025 Data

Walk-forward back testing was conducted on data from January 2025 to January 2026, which was entirely withheld from the training and validation process. The back testing timeline plots for all three metals are presented in Figures 37 through 39, showing daily predicted risk probabilities for both baseline and enhanced models alongside actual risk events marked as red triangles.

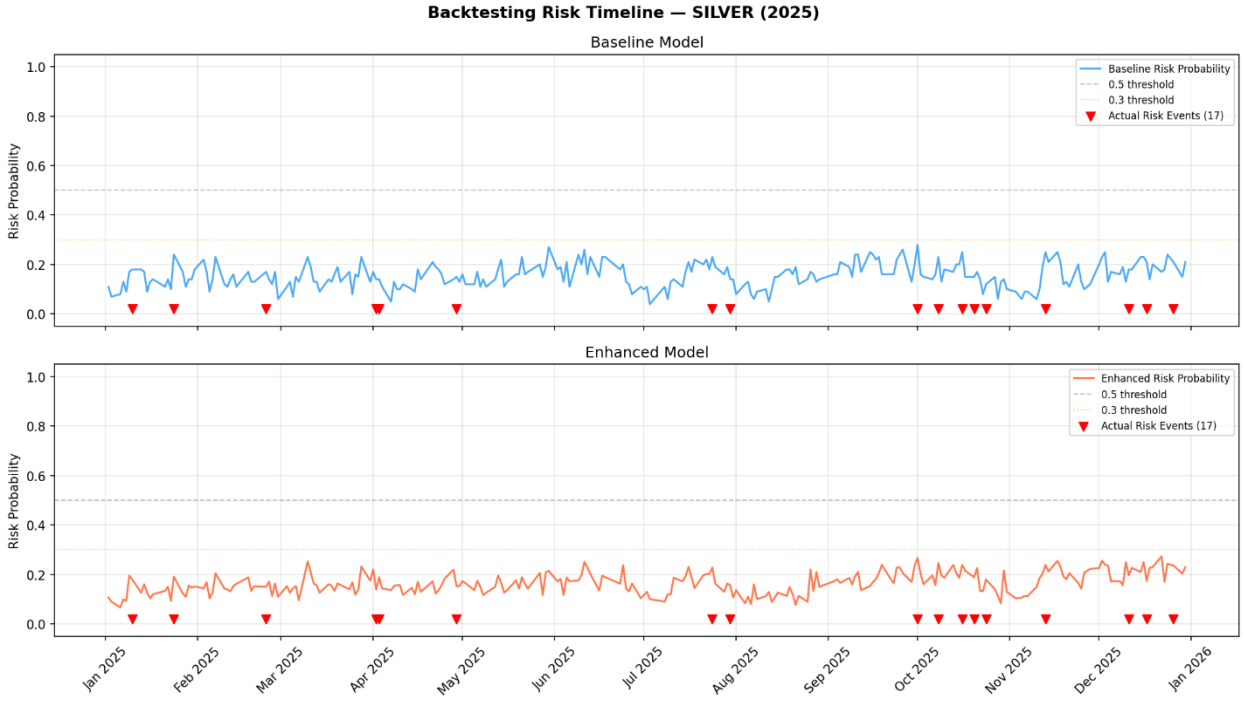


Figure 37: Backtesting risk timeline for silver (2025): baseline and enhanced model daily risk probabilities. Red triangles indicate actual downside risk events (17 total).

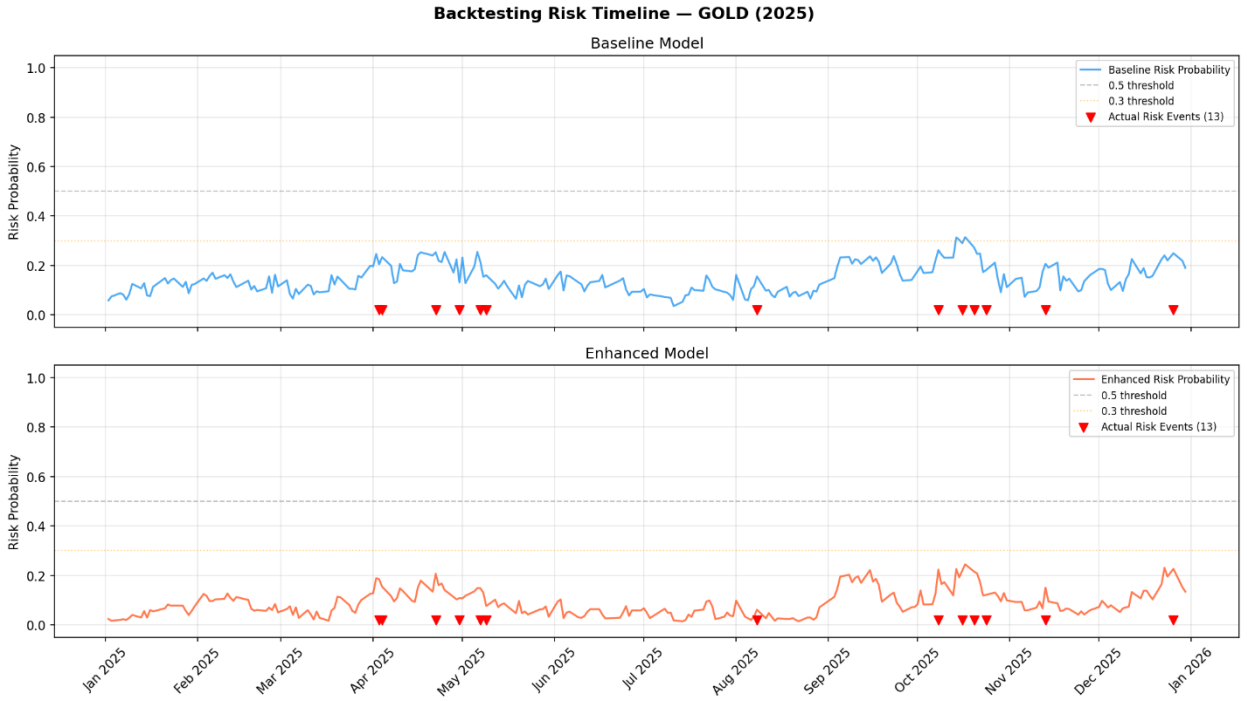


Figure 38: Backtesting risk timeline for gold (2025): baseline model (blue) and enhanced model (orange) daily risk probabilities. Red triangles indicate actual downside risk events (13 total).

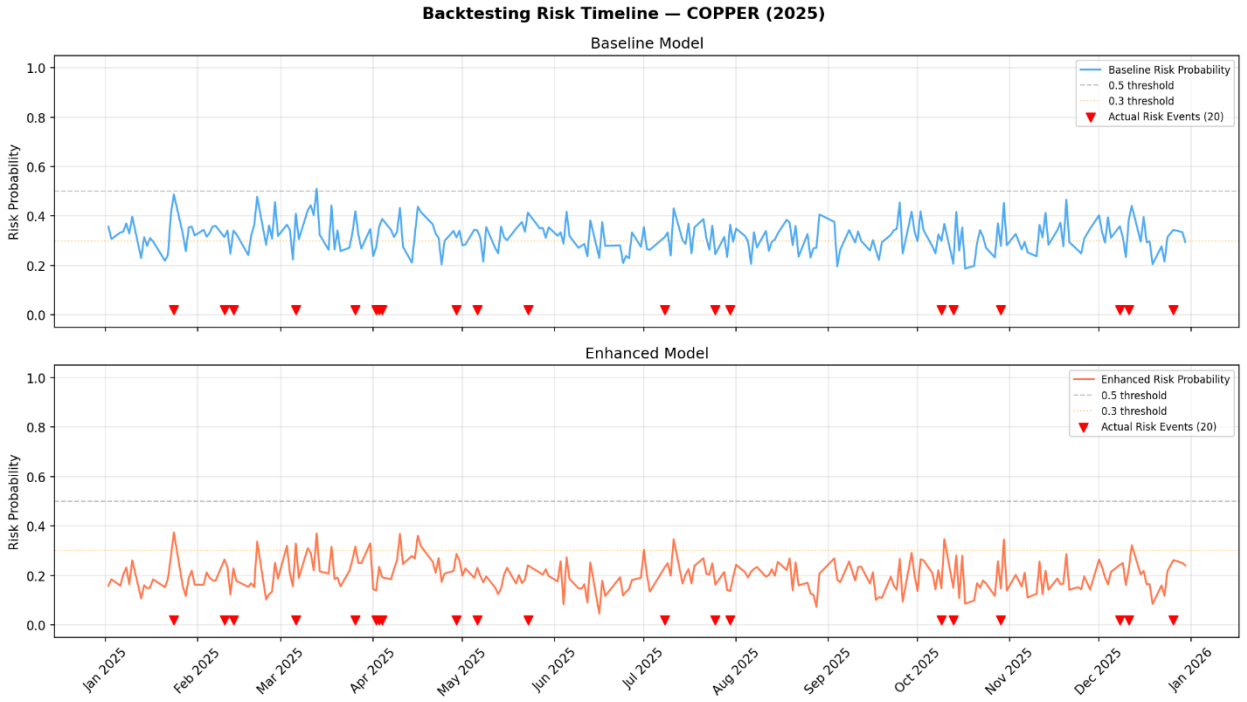


Figure 39: Backtesting risk timeline for copper (2025): baseline and enhanced model daily risk probabilities. Red triangles indicate actual downside risk events (20 total).

Several observations emerge from the backtesting timeline plots. For gold, the baseline model assigns risk probabilities mostly in the 0.05 to 0.25 range throughout 2025. There is a notable rise in predicted risk during October and November 2025, which coincides with

Federal Reserve commentary on interest rate expectations and dollar strength. The enhanced gold model assigns lower probabilities, concentrated between 0.02 and 0.15, reflecting the improved Brier Score calibration. There were 13 actual risk events during the 2025 gold backtest. These events, particularly those in the spring and autumn of 2025, correspond to periods where both models show higher probability elevations compared to non-event periods, indicating a genuine signal in the unseen data.

For silver, the baseline and enhanced models produce similar probability timelines, with risk scores remaining below 0.30 throughout 2025. The 17 actual silver risk events are distributed across the year, with a cluster in October and November 2025 that mirrors the gold pattern. This is consistent with the correlation between precious metal markets during periods of macroeconomic stress.

Copper's backtesting timeline presents the clearest contrast between the models. The baseline model assigns probabilities across a wide range of 0.20 to 0.50, with multiple spikes approaching or exceeding the 0.5 threshold. This represents the highest baseline probabilities observed for any metal in the backtest. By contrast, the enhanced model assigns probabilities in a much narrower range of 0.10 to 0.35. This compression explains the substantial Brier Score improvement from 0.132 to 0.087 observed on the test set.

6 Discussion and Conclusion

6.1 Discussion

The central research question asked whether a Random Forest classifier integrating FinBERT sentiment analysis could accurately predict daily downside risk events in gold, silver, and copper markets while providing interpretable probability-based risk scores and SHAP explanations. The results provide a nuanced answer. The system demonstrates meaningful predictive capability for gold and silver and produces well-calibrated risk probabilities for all three metals. It also generates SHAP explanations consistent with established financial theory, though the contribution of sentiment features is limited by the sparse historical coverage of the sentiment dataset.

6.1.1 Model Performance in Relation to Literature

The ROC-AUC scores for the gold enhanced model (0.752 on the test set and 0.847 on the backtest) align with performance levels in similar financial risk studies. Fischer and Krauss (2018) reported ROC-AUC values between 0.71 and 0.78 for Random Forest models applied to equities. Additionally, a study combining FinBERT with XGBoost achieved an AUC of 0.748 using combined technical and sentiment features. The gold results in this

study sit within that established performance range, which confirms that the experimental design is sound.

The performance improvement achieved by the enhanced model is modest but consistent for gold and silver in terms of ROC-AUC. Furthermore, the calibration gains observed in the Brier Score, particularly for copper, align with findings from Li et al. (2020), who noted that sentiment integration tends to improve probability calibration even when discriminative performance gains are marginal. This study contributes empirical evidence from the coinage metal context, which extends earlier findings to a domain that has received limited attention in the explainable AI literature.

6.1.2 SHAP Interpretability and Alignment with Financial Theory

The SHAP analysis confirms economically interpretable relationships. The 10-year Treasury yield dominates gold with a negative directional contribution, the VIX and Treasury yields co-dominate silver, and S&P 500 returns dominate copper as a procyclical industrial barometer.

However, the absence of sentiment features from the top 20 SHAP rankings is a significant limitation. This is attributable to sparse sentiment coverage rather than a lack of signal quality. Consequently, the project objective of generating SHAP explanations that incorporate sentiment-driven insights was only partially achieved. In future work with more comprehensive sentiment data, sentiment features would likely appear in global importance rankings and contribute to individual prediction explanations, as described by Dhole (2024) and Gu et al. (2024).

6.1.3 Back testing and Real-World Applicability

The back testing results on the 2025 unseen data provide encouraging evidence of generalization, particularly for gold where the enhanced model achieved a ROC-AUC of 0.847. This qualitative validation is consistent with the approach recommended by Giudici et al. (2024), who argue that SHAP-based case analysis of back test predictions provides important supplementary evidence of model reliability beyond quantitative metrics alone.

6.1.4 Limitations

Several limitations must be acknowledged. First, sentiment data is a significant constraint. The NewsAPI free tier limits access to recent headlines, meaning sentiment features were available for less than 3% of training observations.

Second, class imbalance remains a fundamental challenge. The low F1-scores at default thresholds indicate that the models do not generate actionable binary signals without

adjustment. Any practical application would require calibrating the operating threshold based on a specific tolerance for false positives and false negatives.

Third, financial markets are non-stationary, meaning models trained on historical data may not perform well in unprecedented conditions. The strong backtest performance in 2025 is encouraging, but that period shares macroeconomic characteristics with the training window. Consequently, performance in a structurally different macroeconomic regime cannot be guaranteed.

6.2 Conclusion

This project developed an explainable machine learning system to predict short-term downside risk in coinage metal markets. It integrated technical price indicators, macroeconomic features, and sentiment analysis to generate probability-based risk scores supported by SHAP explanations. The system was implemented as a Python pipeline connected to a PostgreSQL database, covering data collection, feature engineering, model training, and backtesting.

The primary limitation is restricted sentiment coverage, which prevented sentiment features from contributing visibly to SHAP global importance rankings despite producing measurable metric improvements. While this does not invalidate the findings, it limits the conclusions that can be drawn regarding the interpretability of sentiment-driven predictions.

The project objectives were met, and the findings contribute to the literature on explainable AI in commodity markets. This study demonstrates that interpretable machine learning frameworks can produce both practically useful risk scores and meaningful feature insights for coinage metal risk prediction.

Future work should prioritize three areas. First, expanding sentiment coverage with a professional data source would allow a more definitive assessment of FinBERT's contribution. Second, threshold calibration studies should identify operating points that balance precision and recall for specific risk management needs. Third, incorporating recurrent neural network components such as LSTM layers would better capture temporal dependencies that the current feature approach only partially addresses.

References

Araci, D. (2019) *FinBERT: Financial sentiment analysis with pre-trained language models*. arXiv preprint arXiv:1908.10063. Available at: https://arxiv.org/abs/1908.10063 (Accessed: 10 November 2025).
Bollen, J., Mao, H. and Zeng, X. (2011) 'Twitter mood predicts the stock market', *Journal of Computational Science*, 2(1). Available at: https://arxiv.org/pdf/1010.3003 (Accessed: 29 September 2025).

Danie, A. et al. (2025) 'Model-agnostic explainable artificial intelligence methods in finance: a systematic review', *Artificial Intelligence Review*, 58. Available at: https://link.springer.com/article/10.1007/s10462-025-11215-9 (Accessed: 12 March 2026).

Doshi-Velez, F. and Kim, B. (2018) 'Towards a rigorous science of interpretable machine learning', *arXiv preprint*. Available at: https://arxiv.org/pdf/1702.08608 (Accessed: 2 October 2025).

Fischer, T. and Krauss, C. (2018) 'Deep learning with long short-term memory networks for financial market predictions', *European Journal of Operational Research*, 270(2). Available at: https://iranarze.ir/wp-content/uploads/2019/01/E10789-IranArze.pdf (Accessed: 17 October 2025).

Giudici, P., Piergallini, A., Recchioni, M.C. and Raffinetti, E. (2024) 'Explainable Artificial Intelligence methods for financial time series', *ScienceDirect*. Available at: https://www.sciencedirect.com/science/article/pii/S037843712400685X (Accessed: 12 November 2025).

Gu, W. et al. (2024) 'Predicting stock prices with FinBERT-LSTM: integrating news sentiment analysis', *Proceedings of the 2024 8th International Conference on Computer Science and Artificial Intelligence*. Available at: https://arxiv.org/pdf/2407.16150 (Accessed: 14 March 2026).

Huang, A., Wang, H. and Yang, Y. (2022) 'FinBERT: a large language model for extracting information from financial text', *Contemporary Accounting Research*.
Jabeur, S.B., Mefteh-Wali, S. and Viviani, J.L. (2024) 'Forecasting gold price with the XGBoost algorithm and SHAP interaction values', *Annals of Operations Research*, 334, pp.679–699. Available at: https://link.springer.com/article/10.1007/s10479-021-04187-w (Accessed: 15 November 2025).

Li, Q., Wang, T. and Wu, J. (2020) 'Financial sentiment analysis for risk prediction using hybrid deep learning', *Expert Systems with Applications*. Available at: https://www.researchgate.net/publication/258821644 (Accessed: 28 October 2025).
Liapis, C.M. and Karanikola, A. (2023) 'Investigating deep stock market forecasting with sentiment analysis', *Entropy*, 25(2), p.219. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC9955765/ (Accessed: 14 March 2026).

Liu, Z., Huang, D., Huang, K., Li, Z. and Zhao, J. (2020) 'FinBERT: a pre-trained financial language representation model for financial text mining', *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. Available at: https://www.ijcai.org/proceedings/2020/0622.pdf (Accessed: 10 November 2025).

Lundberg, S.M. and Lee, S.I. (2017) 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems (NeurIPS)*. Available at: https://arxiv.org/pdf/1705.07874 (Accessed: 2 November 2025).

Mathematics (2025) 'Stock price prediction using FinBERT-enhanced sentiment with SHAP explainability and differential privacy', *Mathematics*, 13(17), p.2747. Available at: https://www.mdpi.com/2227-7390/13/17/2747 (Accessed: 15 March 2026).

Molnar, C. (2022) *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd edn. Independently published. Available at: https://originalstatic.aminer.cn/misc/pdf/Molnar-interpretable-machinelearning_compressed.pdf (Accessed: 9 November 2025).

Monica Hovsepian (2023) 'The benefits and risks of AI in financial services', *Forbes Finance Council*. Available at: https://www.forbes.com/councils/forbesfinancecouncil/2023/12/26/the-benefits-and-risks-of-ai-in-financial-services/ (Accessed: 12 November 2025).

Peng, K. and Yan, G. (2021) 'A survey on deep learning for financial risk prediction', *Quantitative Finance and Economics*, 5(4), pp.716–737. Available at: https://www.aimspress.com/article/doi/10.3934/QFE.2021032 (Accessed: 13 March 2026).

Tuitoek, J. et al. (2025) 'Modelling commodity price co-movement: building on traditional time series models and exploring applications of machine learning algorithms', *Decisions in Economics and Finance*. Available at: https://link.springer.com/article/10.1007/s10203-025-00512-1 (Accessed: 14 March 2026).

Zhang, Y., Liang, M. and Ou, H. (2024) 'Prediction of precious metal index based on ensemble learning and SHAP interpretable method', *Computational Economics*, 64(6), pp.3243–3278.

Appendix A

Semester 1 Weekly Progress Logbook

Weekly Progress Logbook - Semester 1 (Sept 2025 to Jan 2026)			
Student ID:	**33114153**		
Student Name:	**Mohammed Adnan Osman**		
Course:	**Computer Science**		
Project Title:	**Explainable AI for Short-Term Risk Prediction in Coinage Metal Markets.**		
Supervisor Name:	**Dr Nasim Dadashi**		

w/c (Mon)	Week	Weekly log of progress	Supervisor Initials
29/09/2025	1	Reviewed module guide - reviewing range of potential topics for project	M.R
06/10/2025	2	Selected project topic on explainable AI for metal-market risk prediction and reviewed module guide.	M.R
13/10/2025	3	Finalised project title and drafted aims and objectives. - changed from all precious metals to only coinage.	
20/10/2025	4	Started reading papers (5) on Machine Learning & Explainable AI (XAI) in finance.	
27/10/2025	5	Wrote introduction for proposal.	
03/11/2025	6	Began outlining literature review sections. - Read more papers (4) on my Topic.	N.D
10/11/2025	7	Completed literature review and refined research question - Wrote all refernces in Harvard Style.	N.D
17/11/2025	8	Wrote methodology section: planned data sources - Yahoo Finance (API) , tools (Python (ML), SQL, SHAP - XAI).	N.D
24/11/2025	9	Outlined project plan	N.D
01/12/2025	10	Made the Gantt Chart	N.D
08/12/2025	11	Completed full Project Proposal. - slight edit to predict a more than 2% downside risk rather than just decline.	N.D
15/12/2025	12	Completed Ethics Form - Sent Ethics form to supervisor in PDF Format.	N.D
Winter break			
05/01/2026	13	Contacted Dr. Nasim through email for final approval	
12/01/2026	14	Submitting the full Project Proposal with this weekly logbook and the Ethics Form	
19/01/2026	15	Set up PostgreSQL database schema (5 Tables). Started data collection using yFinance API	
26/01/2026	16	Completed data collection and feature engineering scripts. Resolved database errors and verified all data inserted correctly.	

Appendix B

Semester 2 Weekly Progress Logbook

Weekly Progress Logbook - Semester 2 (Feb 2026 to May 2026)	
Student ID:	**33114153**
Student Name:	**Mohammed Adnan Osman**
Course:	**Computer Science**
Project Title:	Explainable AI for Daily Short-Term Risk Prediction in Coinage Metal Markets
Supervisor Name:	Dr Nasim Dadashi

w/c (Mon)	Week	Weekly log of progress	Supervisor Initials
09/02/2026	1	Reviewed semester 1 feedback and planned semester 2 development phases. Confirmed all data collection scripts functioning correctly.	
16/02/2026	**2**	Completed Script 02 feature engineering — RSI, MACD, Bollinger Bands and volume indicators. 4,296 technical feature rows inserted into database.	N.D
23/02/2026	**3**	Completed Script 03 FinBERT sentiment pipeline. 1,321 headlines collected a	N.D
02/03/2026	**3**	Began Script 04 model training. Implemented target variable construction and chronological 60/20/20 split.	N.D
09/03/2026	**9**	Completed Script 04 — trained baseline and enhanced Random Forest model:	N.D
16/03/2026	**8**	Completed Script 05 model evaluation. Generated all evaluation plots and metrics for six model variants.	N.D
23/03/2026	**7**	Completed Script 06 backtesting. Gold enhanced backtest ROC-AUC 0.847. SHAP waterfall plots generated.	N.D
Spring break			
06/04/2026	**8**	database design sections completed.	N.D
13/04/2026	**9**	framework.	N.D
20/04/2026	**10**	Completed Chapter 4 — Scripts 04, 05, 06, 07 documented. Figure numbering fixed throughout.	N.D
27/04/2026	**11**	Reviewed and improved Chapters 5 and 6. Verified all figure references and metric values.	N.D
04/05/2026	12	Final review of all chapters. Appendix and list of figures completed.	
09/05/2026	13		

Appendix C

Model Evaluation Metrics Summary

Table 4: Table C.1 presents the complete evaluation metrics for all six model variants across the held-out test set. Values are reported to four decimal places.

Metal	Model	ROC-AUC	Avg Precision	Brier Score	Log Loss	F1	N_test	N_risk
Gold	Baseline	0.7327	0.2225	0.0562	0.2341	0.0	290	16
Gold	Enhanced	0.7516	0.2443	0.0491	0.1968	0.0	290	16
Silver	Baseline	0.6577	0.1931	0.0716	0.2799	0.0	278	20
Silver	Enhanced	0.6803	0.1657	0.0727	0.2841	0.0	278	20
Copper	Baseline	0.6147	0.1333	0.1319	0.4446	0.0	291	23

Copper	Enhanced	0.6111	0.1657	0.0868	0.3261	0.0	291	23